



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

TECHNICAL DEVELOPMENT MANUAL.

Nombre Completo	N. Cuenta	Grupo de Laboratorio	Grupo de Teoría
Barragan Pilar Diana	318147981	03	04
Medrano Miranda Daniel Ulises	318045351	03	04
Uriarte Ortiz Enrique Yahir	318234757	02	04

TEAM: 04

SEMESTER 2025-1

DEADLINE: Tuesday, November 19, 2024

QUALIFICATION: _____

Technical Development Manual: A manual that explains, through screenshots and code blocks, how the required points in the "Elements to Include within the Scenario" section were covered, along with the section of individual conclusions, group conclusion, and bibliography.

Elements within the Scenario.

- **The board consists of a square area with 10 squares in length, totaling 40 individual squares, each having three different visual states.**

To develop this activity, we followed the original layout from our proposal document, where the corner squares would have a square shape on their top face, while the intermediate squares would be half as wide. As a sketch for the dimensions, we used GeoGebra to visualize how the corner squares would look. To distinguish them from the surface, we defined them with a slight height.

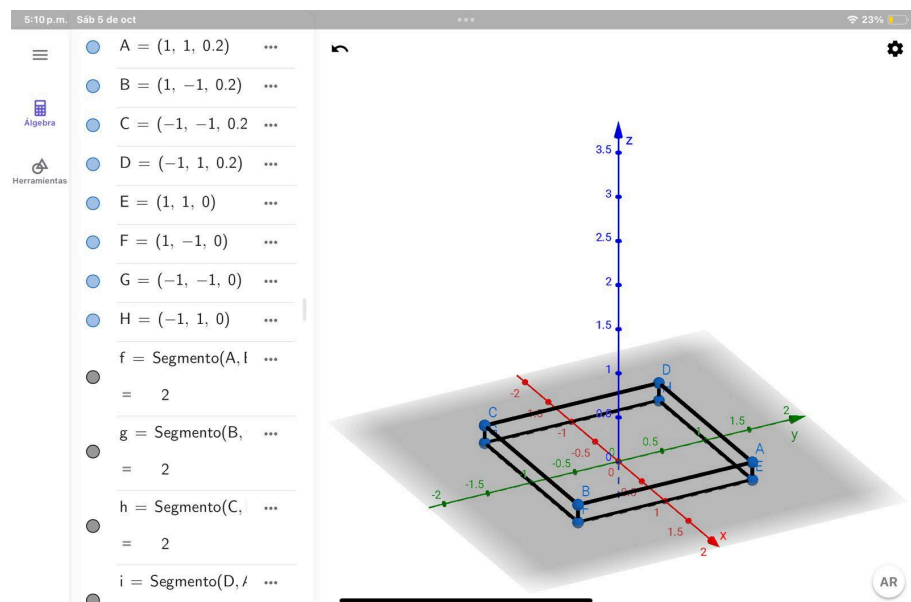


Fig. 1. GeoGebra Sketch of the Corner Squares.

We then moved on to the implementation in the code. First, we defined the creation of the corner squares, which have a total of 23 vertices, with every four vertices defining one face of the rectangular prism. We also defined from the start the direction in which light would reflect on these elements, preparing for the implementation of lighting to add more realism to the rendering.

For the intermediate squares, we developed the same number of vertices, with the difference being that the distance between the top and bottom face vertices was half the distance of the corner squares. The function name for creating corner squares is **void Casillas_Esquina()**, and for the intermediate squares, **void Casillas_Inter()**.

Next, we defined the 40 squares, specifying their position, scale, and rotation. To better identify each one, we used the alias names assigned in our proposal document, labeled with a letter from A to D and a number from 1 to 9. The corner squares follow a different naming code, labeled with the letter S and a

number from 1 to 4.

```
void Casillas_Esquina() {
    unsigned int Casillas1_indices[] = {
        0,1,2, 0,2,3,    //Up
        4,5,6, 4,6,7,    //Down
        8,9,11, 8,10,11,  //Lateral_01
        12,13,15, 12,14,15, //Lateral_02
        16,17,18, 17,18,19, //Lateral_03
        20,21,22, 21,22,23, //Lateral_04
    };

    GLfloat Casillas1_vertices[] = {
        //Up
        1.0f, 0.2f, 1.0f, 0.00f, 1.0f, 0.0f, -1.0f, 0.0f, //A - 0
        1.0f, 0.2f, -1.0f, 0.00f, 0.0f, 0.0f, -1.0f, 0.0f, //B - 1
        -1.0f, 0.2f, -1.0f, 0.79f, 0.0f, 0.0f, -1.0f, 0.0f, //C - 2
        -1.0f, 0.2f, 1.0f, 0.79f, 1.0f, 0.0f, -1.0f, 0.0f, //D - 3

        //Down
        1.0f, 0.0f, 1.0f, 0.79f, 1.0f, 0.0f, 1.0f, 0.0f, //E - 4
        1.0f, 0.0f, -1.0f, 0.79f, 0.0f, 0.0f, 1.0f, 0.0f, //F - 5
        -1.0f, 0.0f, -1.0f, 0.99f, 0.0f, 0.0f, 1.0f, 0.0f, //G - 6
        -1.0f, 0.0f, 1.0f, 0.99f, 1.0f, 0.0f, 1.0f, 0.0f, //H - 7

        //Lateral_01
        1.0f, 0.2f, 1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //A - 8
        1.0f, 0.2f, -1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //B - 9
        1.0f, 0.0f, 1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //E - 10
        1.0f, 0.0f, -1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //F - 11

        //Lateral_02
        1.0f, 0.2f, -1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //B - 12
        -1.0f, 0.2f, -1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //C - 13
        1.0f, 0.0f, -1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //F - 14
        -1.0f, 0.0f, -1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //G - 15

        //Lateral_03
        1.0f, 0.2f, 1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //A - 16
        -1.0f, 0.2f, 1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //D - 17
        1.0f, 0.0f, 1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //E - 18
        -1.0f, 0.0f, 1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //H - 19

        //Lateral_04
        -1.0f, 0.2f, -1.0f, 0.795f, 1.0f, 0.0f, -1.0f, 0.0f, //C - 20
        -1.0f, 0.2f, 1.0f, 0.985f, 1.0f, 0.0f, -1.0f, 0.0f, //D - 21
        -1.0f, 0.0f, -1.0f, 0.795f, 0.0f, 0.0f, -1.0f, 0.0f, //G - 22
        -1.0f, 0.0f, 1.0f, 0.985f, 0.0f, 0.0f, -1.0f, 0.0f, //H - 23
    };

    Mesh* Casillas1 = new Mesh();
    Casillas1->CreateMesh(Casillas1_vertices, Casillas1_indices, 192, 36);
    meshList.push_back(Casillas1);
}
```

Fig.2. Coordinates for Each Corner Square.

```

void Casillas_Inter() {
    unsigned int Casillas2_indices[] = {
        0,1,2, 0,2,3,    //Up
        4,5,6, 4,6,7,    //Down
        8,9,11, 8,10,11,  //Lateral_01
        12,13,15, 12,14,15, //Lateral_02
        16,17,18, 17,18,19, //Lateral_03
        20,21,22, 21,22,23, //Lateral_04
        8,9,11, 8,10,11,  //Lateral_01
        12,13,15, 12,14,15, //Lateral_02
        16,17,18, 17,18,19, //Lateral_03
        20,21,22, 21,22,23, //Lateral_04
    };
    GLfloat Casillas2_vertices[] = {
        //Up
        1.0f, 0.2f, 0.5f, 0.00f,0.0f, 0.0f,-1.0f,0.0f, //A - 0
        1.0f, 0.2f,-0.5f, 0.72f,0.0f, 0.0f,-1.0f,0.0f, //B - 1
        -1.0f, 0.2f,-0.5f, 0.72f,1.0f, 0.0f,-1.0f,0.0f, //C - 2
        -1.0f, 0.2f, 0.5f, 0.00f,1.0f, 0.0f,-1.0f,0.0f, //D - 3

        //Down
        1.0f, 0.0f, 0.5f, 0.75f,1.0f, 0.0f,1.0f,0.0f, //E - 4
        1.0f, 0.0f,-0.5f, 0.75f,0.0f, 0.0f,1.0f,0.0f, //F - 5
        -1.0f, 0.0f,-0.5f, 0.99f,0.0f, 0.0f,1.0f,0.0f, //G - 6
        -1.0f, 0.0f, 0.5f, 0.99f,1.0f, 0.0f,1.0f,0.0f, //H - 7

        //Lateral_01
        1.0f, 0.2f, 0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //A - 8
        1.0f, 0.2f,-0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //B - 9
        1.0f, 0.0f, 0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //E - 10
        1.0f, 0.0f,-0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //F - 11

        //Lateral_02
        1.0f, 0.2f,-0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //B - 12
        -1.0f, 0.2f,-0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //C - 13
        1.0f, 0.0f,-0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //F - 14
        -1.0f, 0.0f,-0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //G - 15

        //Lateral_03
        1.0f, 0.2f, 0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //A - 16
        -1.0f, 0.2f, 0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //D - 17
        1.0f, 0.0f, 0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //E - 18
        -1.0f, 0.0f, 0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //H - 19

        //Lateral_04
        -1.0f, 0.2f,-0.5f, 0.75f,1.0f, 0.0f,-1.0f,0.0f, //C - 20
        -1.0f, 0.2f, 0.5f, 0.985f,1.0f, 0.0f,-1.0f,0.0f, //D - 21
        -1.0f, 0.0f,-0.5f, 0.75f,0.0f, 0.0f,-1.0f,0.0f, //G - 22
        -1.0f, 0.0f, 0.5f, 0.985f,0.0f, 0.0f,-1.0f,0.0f, //H - 23
    };
    Mesh* Casillas2 = new Mesh();
    Casillas2->CreateMesh(Casillas2_vertices, Casillas2_indices, 192, 36);
    meshList.push_back(Casillas2);
}

```

Fig.3. Coordinates for the Intermediate Squares Between the Corner Squares.

Since including a screenshot of the code for all 40 squares would significantly increase the document's size, we are mainly sharing screenshots of the corner squares. The structure for all squares, including the intermediate ones, is nearly identical, as they use the same scale defined by the first placed square, S4. However, for each new direction of the corner squares, we assigned a rotation of 90°. The squares that include this rotation are B1, C1, and D1.

```

int main() {
    mainWindow = Window(1366, 768); // 1280, 1024 or 1024, 768
    mainWindow.Initialise();

    CreateObjects();    Casillas_Esquina();    Casillas_Inter();
    Dado_4();           Dado_8();              Centro();
    CreateShaders();
}

```

Fig.4. Definition of Functions in the Main Function.

```

//Start - Esquina 1
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modellamp1 = model; modellamp2 = model; modellamp3 = model; modellamp4 = model;
model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S4") == 0 && PuedeSubir) { Casilla1_D.UseTexture(); }
else if (FASE_01) { Casilla1.UseTexture(); }
else if (!FASE_01) { Casilla1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//A1
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "A1") == 0 && PuedeSubir) { A1_D.UseTexture(); }
else if (FASE_01) { A1.UseTexture(); }
else if (!FASE_01) { A1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.5. Definition of Casillas S4 and A1.

```

//A9
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "A9") == 0 && PuedeSubir) { A9_D.UseTexture(); }
else if (FASE_01) { A9.UseTexture(); }
else if (!FASE_01) { A9_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

//Esquina 2
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S1") == 0 && PuedeSubir) { Csl2_D.UseTexture(); }
else if (FASE_01) { Csl2.UseTexture(); }
else if (!FASE_01) { Csl2_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//B1
model = glm::translate(model, glm::vec3(-1.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "B1") == 0 && PuedeSubir) { B1_D.UseTexture(); }
else if (FASE_01) { B1.UseTexture(); }
else if (!FASE_01) { B1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.6. Definition of Casillas A9, S1, and B1.

```

//B9
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "B9") == 0 && PuedeSubir) { B9_D.UseTexture(); }
else if (FASE_01) { B9.UseTexture(); }
else if (!FASE_01) { B9_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

//Esquina 3
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S2") == 0 && PuedeSubir) { Csl3_D.UseTexture(); }
else if (FASE_01) { Csl3.UseTexture(); }
else if (!FASE_01) { Csl3_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//C1
model = glm::translate(model, glm::vec3(1.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "C1") == 0 && PuedeSubir) { C1_D.UseTexture(); }
else if (FASE_01) { C1.UseTexture(); }
else if (!FASE_01) { C1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.7. Definition of Casillas B9, S2, and C1.

```

//C9
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "C9") == 0 && PuedeSubir) { C9_D.UseTexture(); }
else if (FASE_01) { C9.UseTexture(); }
else if (!FASE_01) { C9_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

//Esquina 4
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "S3") == 0 && PuedeSubir) { Casilla4_D.UseTexture(); }
else if (FASE_01) { Casilla4.UseTexture(); }
else if (!FASE_01) { Casilla4_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();

//D1
model = glm::translate(model, glm::vec3(-1.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
if (strcmp(casilla_Activacion, "D1") == 0 && PuedeSubir) { D1_D.UseTexture(); }
else if (FASE_01) { D1.UseTexture(); }
else if (!FASE_01) { D1_N.UseTexture(); }
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[5]->RenderMesh();

```

Fig.8. Definition of Casillas C9, S3, and D1.

- **Apply 3 different textures to the squares:**
 - **Base texture when we are in the day cycle.**
 - **Base texture when we are in the night cycle.**
 - **Base texture when we are on the square, regardless of the cycle.**

To implement this texture change, we first defined the textures for each square. The structure of all of them is as follows: each has the name of the character, an image of the character, and a background color. The squares that are active during the day cycle are named according to the square, as explained in the previous implementation, and are colored yellow-orange. The illuminated textures have "_D" added to the name and are colored aqua blue. The textures for the night cycle have "_N" added to the name and are colored purple. Additionally, the corner squares have a stronger tone of these colors to differentiate them further. To avoid increasing the document size, we only show the screenshot of the texture for square S1. All textures were created with the help of the GIMP program.

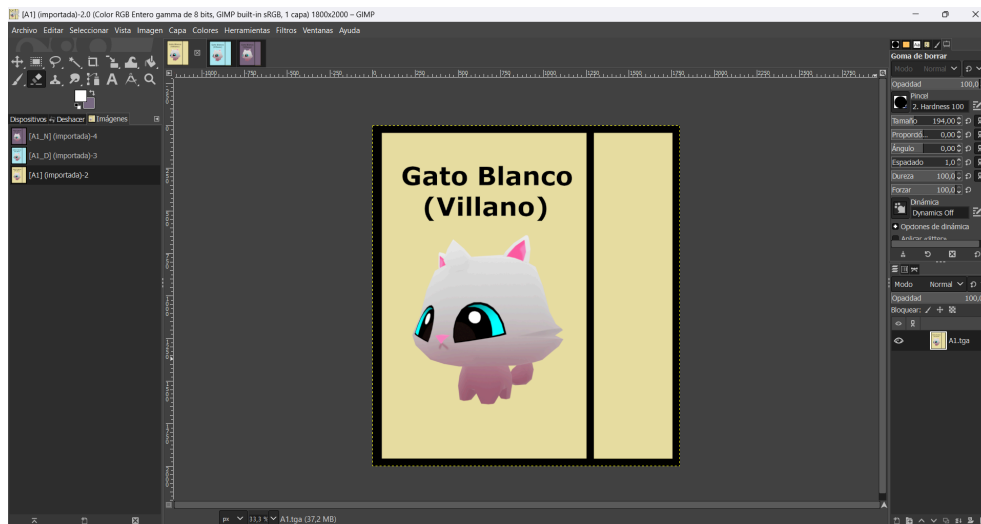


Fig.9. Day texture for square A1.

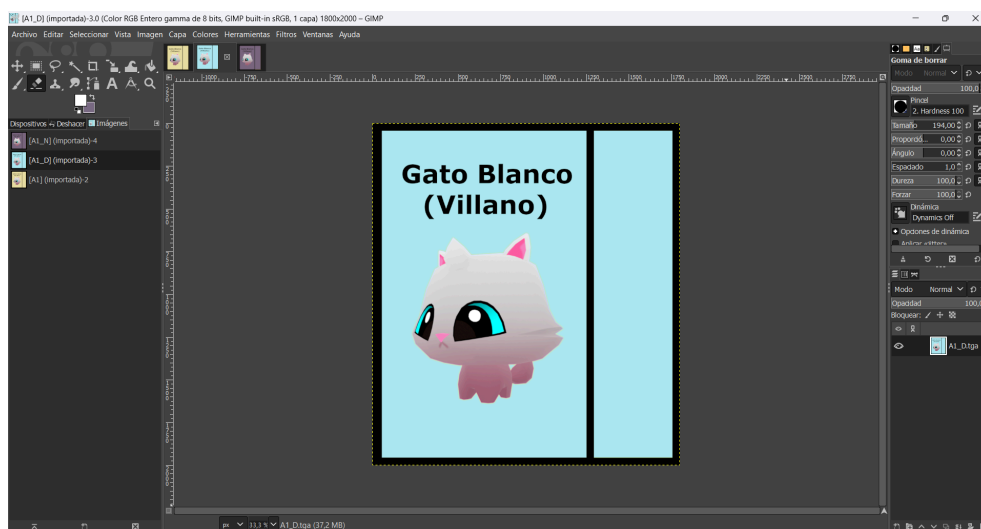


Fig.10. Illuminated texture for square A1.

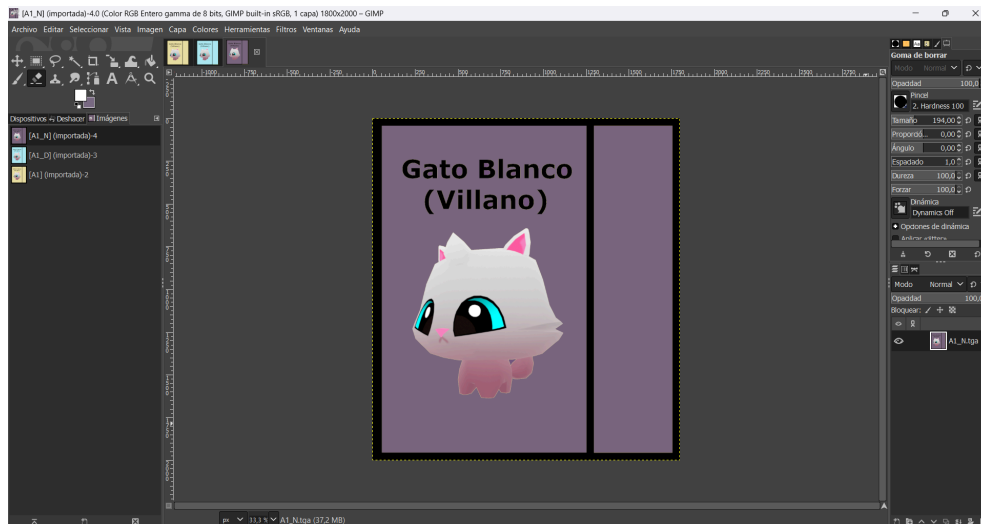


Fig.11. Night texture for square A1.

To define the areas where each square should take the texture, we assigned the location of each vertex. This is why the design of the square textures includes a square and then a rectangle, as the rectangle is for the other faces, while the top face of the shapes is the only one that will display the character's image. The assignment of coordinates according to the texture can be seen in the images **Fig. 2** and **Fig. 3**.

Within the project code, we began defining each texture, as well as its location within the texture folder.

```
//Texturas para las esquinas

//DIA
Casilla1 = Texture("Texturas/Casilla_1.tga");    Casilla1.LoadTextureA();
Csl2 = Texture("Texturas/Casilla_2.tga");        Csl2.LoadTextureA();
Csl3 = Texture("Texturas/Casilla_3.tga");        Csl3.LoadTextureA();
Casilla4 = Texture("Texturas/Casilla_4.tga");    Casilla4.LoadTextureA();
//ILUMINADOS
Casilla1_D = Texture("Texturas/Casilla_1_D.tga"); Casilla1_D.LoadTextureA();
Csl2_D = Texture("Texturas/Casilla_2_D.tga");    Csl2_D.LoadTextureA();
Csl3_D = Texture("Texturas/Casilla_3_D.tga");    Csl3_D.LoadTextureA();
Casilla4_D = Texture("Texturas/Casilla_4_D.tga"); Casilla4_D.LoadTextureA();
//NOCHE
Casilla1_N = Texture("Texturas/Casilla_1_N.tga"); Casilla1_N.LoadTextureA();
Csl2_N = Texture("Texturas/Casilla_2_N.tga");    Csl2_N.LoadTextureA();
Csl3_N = Texture("Texturas/Casilla_3_N.tga");    Csl3_N.LoadTextureA();
Casilla4_N = Texture("Texturas/Casilla_4_N.tga"); Casilla4_N.LoadTextureA();
```

Fig.12. Corner texture, for day, illuminated, and night.

```
//Texturas para las 40 Casillas
//DIA
A1 = Texture("Texturas/A1.tga");    A1.LoadTextureA();    A2 = Texture("Texturas/A2.tga");    A2.LoadTextureA();
A3 = Texture("Texturas/A3.tga");    A3.LoadTextureA();    A4 = Texture("Texturas/A4.tga");    A4.LoadTextureA();
A5 = Texture("Texturas/A5.tga");    A5.LoadTextureA();    A6 = Texture("Texturas/A6.tga");    A6.LoadTextureA();
A7 = Texture("Texturas/A7.tga");    A7.LoadTextureA();    A8 = Texture("Texturas/A8.tga");    A8.LoadTextureA();
A9 = Texture("Texturas/A9.tga");    A9.LoadTextureA();    B1 = Texture("Texturas/B1.tga");    B1.LoadTextureA();
B2 = Texture("Texturas/B2.tga");    B2.LoadTextureA();    B3 = Texture("Texturas/B3.tga");    B3.LoadTextureA();
B4 = Texture("Texturas/B4.tga");    B4.LoadTextureA();    B5 = Texture("Texturas/B5.tga");    B5.LoadTextureA();
B6 = Texture("Texturas/B6.tga");    B6.LoadTextureA();    B7 = Texture("Texturas/B7.tga");    B7.LoadTextureA();
B8 = Texture("Texturas/B8.tga");    B8.LoadTextureA();    B9 = Texture("Texturas/B9.tga");    B9.LoadTextureA();
C1 = Texture("Texturas/C1.tga");    C1.LoadTextureA();    C2 = Texture("Texturas/C2.tga");    C2.LoadTextureA();
C3 = Texture("Texturas/C3.tga");    C3.LoadTextureA();    C4 = Texture("Texturas/C4.tga");    C4.LoadTextureA();
C5 = Texture("Texturas/C5.tga");    C5.LoadTextureA();    C6 = Texture("Texturas/C6.tga");    C6.LoadTextureA();
C7 = Texture("Texturas/C7.tga");    C7.LoadTextureA();    C8 = Texture("Texturas/C8.tga");    C8.LoadTextureA();
C9 = Texture("Texturas/C9.tga");    C9.LoadTextureA();    D1 = Texture("Texturas/D1.tga");    D1.LoadTextureA();
D2 = Texture("Texturas/D2.tga");    D2.LoadTextureA();    D3 = Texture("Texturas/D3.tga");    D3.LoadTextureA();
D4 = Texture("Texturas/D4.tga");    D4.LoadTextureA();    D5 = Texture("Texturas/D5.tga");    D5.LoadTextureA();
D6 = Texture("Texturas/D6.tga");    D6.LoadTextureA();    D7 = Texture("Texturas/D7.tga");    D7.LoadTextureA();
D8 = Texture("Texturas/D8.tga");    D8.LoadTextureA();    D9 = Texture("Texturas/D9.tga");    D9.LoadTextureA();
```

Fig.13. Texture of all intermediate squares, for day.

```
//ILUMINADOS
A1_D = Texture("Textures/A1_D.tga"); A1_D.LoadTextureA(); A2_D = Texture("Textures/A2_D.tga"); A2_D.LoadTextureA();
A3_D = Texture("Textures/A3_D.tga"); A3_D.LoadTextureA(); A4_D = Texture("Textures/A4_D.tga"); A4_D.LoadTextureA();
A5_D = Texture("Textures/A5_D.tga"); A5_D.LoadTextureA(); A6_D = Texture("Textures/A6_D.tga"); A6_D.LoadTextureA();
A7_D = Texture("Textures/A7_D.tga"); A7_D.LoadTextureA(); A8_D = Texture("Textures/A8_D.tga"); A8_D.LoadTextureA();
A9_D = Texture("Textures/A9_D.tga"); A9_D.LoadTextureA(); B1_D = Texture("Textures/B1_D.tga"); B1_D.LoadTextureA();
B2_D = Texture("Textures/B2_D.tga"); B2_D.LoadTextureA(); B3_D = Texture("Textures/B3_D.tga"); B3_D.LoadTextureA();
B4_D = Texture("Textures/B4_D.tga"); B4_D.LoadTextureA(); B5_D = Texture("Textures/B5_D.tga"); B5_D.LoadTextureA();
B6_D = Texture("Textures/B6_D.tga"); B6_D.LoadTextureA(); B7_D = Texture("Textures/B7_D.tga"); B7_D.LoadTextureA();
B8_D = Texture("Textures/B8_D.tga"); B8_D.LoadTextureA(); B9_D = Texture("Textures/B9_D.tga"); B9_D.LoadTextureA();
C1_D = Texture("Textures/C1_D.tga"); C1_D.LoadTextureA(); C2_D = Texture("Textures/C2_D.tga"); C2_D.LoadTextureA();
C3_D = Texture("Textures/C3_D.tga"); C3_D.LoadTextureA(); C4_D = Texture("Textures/C4_D.tga"); C4_D.LoadTextureA();
C5_D = Texture("Textures/C5_D.tga"); C5_D.LoadTextureA(); C6_D = Texture("Textures/C6_D.tga"); C6_D.LoadTextureA();
C7_D = Texture("Textures/C7_D.tga"); C7_D.LoadTextureA(); C8_D = Texture("Textures/C8_D.tga"); C8_D.LoadTextureA();
C9_D = Texture("Textures/C9_D.tga"); C9_D.LoadTextureA(); D1_D = Texture("Textures/D1_D.tga"); D1_D.LoadTextureA();
D2_D = Texture("Textures/D2_D.tga"); D2_D.LoadTextureA(); D3_D = Texture("Textures/D3_D.tga"); D3_D.LoadTextureA();
D4_D = Texture("Textures/D4_D.tga"); D4_D.LoadTextureA(); D5_D = Texture("Textures/D5_D.tga"); D5_D.LoadTextureA();
D6_D = Texture("Textures/D6_D.tga"); D6_D.LoadTextureA(); D7_D = Texture("Textures/D7_D.tga"); D7_D.LoadTextureA();
D8_D = Texture("Textures/D8_D.tga"); D8_D.LoadTextureA(); D9_D = Texture("Textures/D9_D.tga"); D9_D.LoadTextureA();
```

Fig.14. Texture of all intermediate squares, illuminated.

```
//NOCHE
A1_N = Texture("Textures/A1_N.tga"); A1_N.LoadTextureA(); A2_N = Texture("Textures/A2_N.tga"); A2_N.LoadTextureA();
A3_N = Texture("Textures/A3_N.tga"); A3_N.LoadTextureA(); A4_N = Texture("Textures/A4_N.tga"); A4_N.LoadTextureA();
A5_N = Texture("Textures/A5_N.tga"); A5_N.LoadTextureA(); A6_N = Texture("Textures/A6_N.tga"); A6_N.LoadTextureA();
A7_N = Texture("Textures/A7_N.tga"); A7_N.LoadTextureA(); A8_N = Texture("Textures/A8_N.tga"); A8_N.LoadTextureA();
A9_N = Texture("Textures/A9_N.tga"); A9_N.LoadTextureA(); B1_N = Texture("Textures/B1_N.tga"); B1_N.LoadTextureA();
B2_N = Texture("Textures/B2_N.tga"); B2_N.LoadTextureA(); B3_N = Texture("Textures/B3_N.tga"); B3_N.LoadTextureA();
B4_N = Texture("Textures/B4_N.tga"); B4_N.LoadTextureA(); B5_N = Texture("Textures/B5_N.tga"); B5_N.LoadTextureA();
B6_N = Texture("Textures/B6_N.tga"); B6_N.LoadTextureA(); B7_N = Texture("Textures/B7_N.tga"); B7_N.LoadTextureA();
B8_N = Texture("Textures/B8_N.tga"); B8_N.LoadTextureA(); B9_N = Texture("Textures/B9_N.tga"); B9_N.LoadTextureA();
C1_N = Texture("Textures/C1_N.tga"); C1_N.LoadTextureA(); C2_N = Texture("Textures/C2_N.tga"); C2_N.LoadTextureA();
C3_N = Texture("Textures/C3_N.tga"); C3_N.LoadTextureA(); C4_N = Texture("Textures/C4_N.tga"); C4_N.LoadTextureA();
C5_N = Texture("Textures/C5_N.tga"); C5_N.LoadTextureA(); C6_N = Texture("Textures/C6_N.tga"); C6_N.LoadTextureA();
C7_N = Texture("Textures/C7_N.tga"); C7_N.LoadTextureA(); C8_N = Texture("Textures/C8_N.tga"); C8_N.LoadTextureA();
C9_N = Texture("Textures/C9_N.tga"); C9_N.LoadTextureA(); D1_N = Texture("Textures/D1_N.tga"); D1_N.LoadTextureA();
D2_N = Texture("Textures/D2_N.tga"); D2_N.LoadTextureA(); D3_N = Texture("Textures/D3_N.tga"); D3_N.LoadTextureA();
D4_N = Texture("Textures/D4_N.tga"); D4_N.LoadTextureA(); D5_N = Texture("Textures/D5_N.tga"); D5_N.LoadTextureA();
D6_N = Texture("Textures/D6_N.tga"); D6_N.LoadTextureA(); D7_N = Texture("Textures/D7_N.tga"); D7_N.LoadTextureA();
D8_N = Texture("Textures/D8_N.tga"); D8_N.LoadTextureA(); D9_N = Texture("Textures/D9_N.tga"); D9_N.LoadTextureA();
```

Fig.15. Texture of all intermediate squares, for night.

To control the texture change, we defined a boolean variable called **FASE_01**. The purpose of this variable is to set it to **true** when it is daytime, and **false** when it is nighttime. The change to an illuminated texture depends on two variables that were defined later on. The first is a character-type code, whose value changes to the code of the square in which the character is located, called **casilla_Activación**. The second variable indicates when the character model of the square can be elevated. This ensures that the illuminated texture change only occurs once the character reaches the square. Later, it will be explained in detail how these variables obtain their values. In the images **Fig. 5**, **Fig. 6**, **Fig. 7**, and **Fig. 8**, this implementation is shown before the assignment of opaque material and the implemented model.

```
// --- Iluminacion ---
float T_Transcurrido; bool FASE_01;
const float TIEMPO_CAMBIO = 1000.0f;
```

Fig.16. Definition of the Variable to Change Textures.

- **Special Interaction.** When the avatar completes the path on the square, a 3D model of the element displayed in the square's base texture will appear next to the square. The model will rise from below the board, moving upward and rotating. It will be displayed for 3 seconds and then "hide" by descending.

To implement this functionality, we defined variables for these movements, starting with a rotation variable for each model, as it is through these that we will later make the model rise while rotating.

```
// Variables para la Interacción de los modelos por movimiento.
float ultimaActivacion; bool avanza2;
float rotA1 = 0.0f, rotA2 = 0.0f, rotA3 = 0.0f, rotA4 = 0.0f, rotA5 = 0.0f, rotA6 = 0.0f, rotA7 = 0.0f, rotA8 = 0.0f, rotA9 = 0.0f, rotS1 = 0.0f;
float rotB1 = 0.0f, rotB2 = 0.0f, rotB3 = 0.0f, rotB4 = 0.0f, rotB5 = 0.0f, rotB6 = 0.0f, rotB7 = 0.0f, rotB8 = 0.0f, rotB9 = 0.0f, rotS2 = 0.0f;
float rotC1 = 0.0f, rotC2 = 0.0f, rotC3 = 0.0f, rotC4 = 0.0f, rotC5 = 0.0f, rotC6 = 0.0f, rotC7 = 0.0f, rotC8 = 0.0f, rotC9 = 0.0f, rotS3 = 0.0f;
float rotD1 = 0.0f, rotD2 = 0.0f, rotD3 = 0.0f, rotD4 = 0.0f, rotD5 = 0.0f, rotD6 = 0.0f, rotD7 = 0.0f, rotD8 = 0.0f, rotD9 = 0.0f;
```

Fig.17. Definition of the rotation variables for each model.

Next, we defined the variables to be used to achieve the movement of the models. First, movY[nombre_casilla], which are all the variables we will use to raise the model by moving it along the Y-axis. Then, we defined character array-type variables to determine that the last square implemented is the one we want. We also added flag variables to track when each model progresses and where it stops. Finally, we stored the time of the last activation of the square.

We performed the same variable definition for each model in its respective universe.

```
// --- Casillas de la CHICAS SUPERPODEROSAS ---
float movYA1, movYA2, movYA3, movYA4, movYA5, movYA6, movYA7, movYA8, movYA9, movYS1, movYB1, movYB2, movYB3;
static char ultimaCasillaGato[3] = ""; bool avanzaGato = true; float ultimaActivacionGato = 0.0f;
static char ultimaCasillaOcti[3] = ""; bool avanzaOcti = true; float ultimaActivacionOcti = 0.0f;
static char ultimaCasillaArbol[3] = ""; bool avanzaArbol = true; float ultimaActivacionArbol = 0.0f;
static char ultimaCasillaBurbuja[3] = ""; bool avanzaBurbuja = true; float ultimaActivacionBurbuja = 0.0f;
static char ultimaCasillaBellota[3] = ""; bool avanzaBellota = true; float ultimaActivacionBellota = 0.0f;
static char ultimaCasillaMojoJojo[3] = ""; bool avanzaMojoJojo = true; float ultimaActivacionMojoJojo = 0.0f;
static char ultimaCasillaPeludito[3] = ""; bool avanzaPeludito = true; float ultimaActivacionPeludito = 0.0f;
static char ultimaCasillaPrincesa[3] = ""; bool avanzaPrincesa = true; float ultimaActivacionPrincesa = 0.0f;
static char ultimaCasillaProfesor[3] = ""; bool avanzaProfesor = true; float ultimaActivacionProfesor = 0.0f;
static char ultimaCasillaSedusa[3] = ""; bool avanzaSedusa = true; float ultimaActivacionSedusa = 0.0f;
static char ultimaCasillaCasaCSP[3] = ""; bool avanzaCasaCSP = true; float ultimaActivacionCasaCSP = 0.0f;
static char ultimaCasillaEdificioCSP[3] = ""; bool avanzaEdificioCSP = true; float ultimaActivacionEdificioCSP = 0.0f;
static char ultimaCasillaAlcaldia[3] = ""; bool avanzaAlcaldia = true; float ultimaActivacionAlcaldia = 0.0f;
```

Fig.18. Definition of the Y-axis movement variables.

```
// --- Casillas de BIG HERO 6 ---
float movYB4, movYB5, movYB6, movYB7, movYB8, movYB9, movYS2, movYC1, movYC2, movYC3, movYC4, movYC5, movYC6;
static char ultimaCasillaHiroHamada_M[3] = ""; bool avanzaHiroHamada_M = true; float ultimaActivacionHiroHamada_M = 0.0f;
static char ultimaCasillaPlanta1_M[3] = ""; bool avanzaPlanta1_M = true; float ultimaActivacionPlanta1_M = 0.0f;
static char ultimaCasillaAutoChico_M[3] = ""; bool avanzaAutoChico_M = true; float ultimaActivacionAutoChico_M = 0.0f;
static char ultimaCasillaGogoTomago_M[3] = ""; bool avanzaGogoTomago_M = true; float ultimaActivacionGogoTomago_M = 0.0f;
static char ultimaCasillaMotocicleta_M[3] = ""; bool avanzaMotocicleta_M = true; float ultimaActivacionMotocicleta_M = 0.0f;
static char ultimaCasillaGoldenGate_M[3] = ""; bool avanzaGoldenGate_M = true; float ultimaActivacionGoldenGate_M = 0.0f;
static char ultimaCasillaPlanta2_M[3] = ""; bool avanzaPlanta2_M = true; float ultimaActivacionPlanta2_M = 0.0f;
static char ultimaCasillaBaymaxSuperArmadura_M[3] = ""; bool avanzaBaymaxSuperArmadura_M = true; float ultimaActivacionBaymaxSuperArmadura_M = 0.0f;
static char ultimaCasillaAutoGrande_M[3] = ""; bool avanzaAutoGrande_M = true; float ultimaActivacionAutoGrande_M = 0.0f;
static char ultimaCasillaFred_M[3] = ""; bool avanzaFred_M = true; float ultimaActivacionFred_M = 0.0f;
static char ultimaCasillaPlanta3_M[3] = ""; bool avanzaPlanta3_M = true; float ultimaActivacionPlanta3_M = 0.0f;
static char ultimaCasillaCasaHiro_M[3] = ""; bool avanzaCasaHiro_M = true; float ultimaActivacionCasaHiro_M = 0.0f;
static char ultimaCasillaMegabot_M[3] = ""; bool avanzaMegabot_M = true; float ultimaActivacionMegabot_M = 0.0f;
```

Fig.19. Definition of the Y-axis movement variables.

```
// --- Casillas de PHINEAS Y FERB ---
float movYC7, movYC8, movYC9, movYS3, movYD1, movYD2, movYD3, movYD4, movYD5, movYD6, movYD7, movYD8, movYD9;
static char ultimaCasillaPhineas[3] = ""; bool avanzaPhineas = true; float ultimaActivacionPhineas = 0.0f;
static char ultimaCasillaOrnitocymborg[3] = ""; bool avanzaOrnitocymborg = true; float ultimaActivacionOrnitocymborg = 0.0f;
static char ultimaCasillaIsabella[3] = ""; bool avanzaIsabella = true; float ultimaActivacionIsabella = 0.0f;
static char ultimaCasillaEdoofenshmirtz[3] = ""; bool avanzaEdoofenshmirtz = true; float ultimaActivacionEdoofenshmirtz = 0.0f;
static char ultimaCasillaPeterPanda[3] = ""; bool avanzaPeterPanda = true; float ultimaActivacionPeterPanda = 0.0f;
static char ultimaCasillaDoofenshmirtz[3] = ""; bool avanzaDoofenshmirtz = true; float ultimaActivacionDoofenshmirtz = 0.0f;
static char ultimaCasillaTerryTortuga[3] = ""; bool avanzaTerryTortuga = true; float ultimaActivacionTerryTortuga = 0.0f;
static char ultimaCasillaCandace[3] = ""; bool avanzaCandace = true; float ultimaActivacionCandace = 0.0f;
static char ultimaCasillaDoofenshmirtz2D[3] = ""; bool avanzaDoofenshmirtz2D = true; float ultimaActivacionDoofenshmirtz2D = 0.0f;
static char ultimaCasillaBuford[3] = ""; bool avanzaBuford = true; float ultimaActivacionBuford = 0.0f;
static char ultimaCasillaPinkyChihuahua[3] = ""; bool avanzaPinkyChihuahua = true; float ultimaActivacionPinkyChihuahua = 0.0f;
static char ultimaCasillaCarl[3] = ""; bool avanzaCarl = true; float ultimaActivacionCarl = 0.0f;
static char ultimaCasillaFerb[3] = ""; bool avanzaFerb = true; float ultimaActivacionFerb = 0.0f;
```

Fig.20. Definition of the Y-axis movement variables.

```
// Modelos alrededor del tablero
Model Gato, Octi, Arbol, Burbuja, Bellota, Mojo, Peludito, Princesa, Profesor, Sedusa, Casa, Edificio, Alcaldia, Phineas, Ornitocborg,
Isabella, PeterPanda, Doofenshmirtz, TerryTortuga, Candace, Doofenshmirtz2D, Buford, PinkyChihuahua, Carl, Ferb,
HiroHamada_M, Planta1_M, AutoChico_M, GogoTomago_M, Motocicleta_M, GoldenGate_M, Planta2_M, BaymaxSuperArmadura_M, AutoGrande_M,
Fred_M, Planta3_M, CasaHiro_M, Megabot_M, Lampara, Lampara2, esf;
```

Fig.21. Definition of the models that will be around the board.

```
Gato = Model();
Octi = Model();
Arbol = Model();
Burbuja = Model();
Bellota = Model();
Mojo = Model();
Peludito = Model();
Princesa = Model();
Profesor = Model();
Sedusa = Model();
Casa = Model();
Edificio = Model();
Alcaldia = Model();
HiroHamada_M = Model();
Planta1_M = Model();
AutoChico_M = Model();
GogoTomago_M = Model();
Motocicleta_M = Model();
GoldenGate_M = Model();
Planta2_M = Model();
BaymaxSuperArmadura_M = Model();
AutoGrande_M = Model();
Fred_M = Model();
Planta3_M = Model();
CasaHiro_M = Model();
Megabot_M = Model();
Phineas = Model();
Ornitocborg = Model();
Isabella = Model();
PeterPanda = Model();
Doofenshmirtz = Model();
TerryTortuga = Model();
Candace = Model();
Doofenshmirtz2D = Model();
Buford = Model();
PinkyChihuahua = Model();
Carl = Model();
Ferb = Model();
Lampara = Model();
Lampara2 = Model();

Gato.LoadModel("Models/gato.obj");
Octi.LoadModel("Models/octi.obj");
Arbol.LoadModel("Models/arboll.obj");
Burbuja.LoadModel("Models/burbuja.obj");
Bellota.LoadModel("Models/bellota.obj");
Mojo.LoadModel("Models/mojo.obj");
Peludito.LoadModel("Models/peludito.obj");
Princesa.LoadModel("Models/princesa.obj");
Profesor.LoadModel("Models/profesor.obj");
Sedusa.LoadModel("Models/sedusa.obj");
Casa.LoadModel("Models/casa.obj");
Edificio.LoadModel("Models/edificio.obj");
Alcaldia.LoadModel("Models/alcaldia.obj");
HiroHamada_M.LoadModel("Models/HiroHamada/hiro.obj");
Planta1_M.LoadModel("Models/Planta1/Phormium_1.obj");
AutoChico_M.LoadModel("Models/AutoChico/isetta.obj");
GogoTomago_M.LoadModel("Models/GogoTomago/gogo.obj");
Motocicleta_M.LoadModel("Models/Motocicleta/motocicleta.obj");
GoldenGate_M.LoadModel("Models/GoldenGate/golden_gate_bridge.obj");
Planta2_M.LoadModel("Models/Planta2/hoewa_Forsteriana_1.obj");
BaymaxSuperArmadura_M.LoadModel("Models/BaymaxSuperArmadura/baymax_super.obj");
AutoGrande_M.LoadModel("Models/AutoGrande/500_fiat_lod_a.obj");
Fred_M.LoadModel("Models/Fred/fred.obj");
Planta3_M.LoadModel("Models/Planta3/Planta3.obj");
CasaHiro_M.LoadModel("Models/CasaHiro/CasaHiro.obj");
Megabot_M.LoadModel("Models/Megabot/MegaBot.obj");
Phineas.LoadModel("Models/PHINEAS.obj");
Ornitocborg.LoadModel("Models/Ornitocborg.obj");
Isabella.LoadModel("Models/Isabella.obj");
PeterPanda.LoadModel("Models/PeterPanda.obj");
Doofenshmirtz.LoadModel("Models/Doofenshmirtz.obj");
TerryTortuga.LoadModel("Models/TerryTortuga.obj");
Candace.LoadModel("Models/Candace.obj");
Doofenshmirtz2D.LoadModel("Models/Doofenshmirtz2D.obj");
Buford.LoadModel("Models/Buford.obj");
PinkyChihuahua.LoadModel("Models/PinkyChihuahua.obj");
Carl.LoadModel("Models/Carl.obj");
Ferb.LoadModel("Models/Ferb.obj");
Lampara.LoadModel("Models/fobj_lamp_MODFIC.obj");
Lampara2.LoadModel("Models/obj_0401_street_lamp.obj");
```

Fig.21. We imported and specified the path of the models around the board.

```
// Variables para la Interacción de los modelos por movimiento.
ultimaActivacion = 0.0f; avanza2 = true;
movYA1 = 0.0f; movYA2 = 0.0f; movYA3 = 0.0f; movYA4 = 0.0f; movYA5 = 0.0f; movYA6 = 0.0f; movYA7 = 0.0f; movYA8 = 0.0f; movYA9 = 0.0f; movYS1 = 0.0f;
movVB1 = 0.0f; movVB2 = 0.0f; movVB3 = 0.0f; movVB4 = 0.0f; movVB5 = 0.0f; movVB6 = 0.0f; movVB7 = 0.0f; movVB8 = 0.0f; movVB9 = 0.0f; movYS2 = 0.0f;
movYC1 = 0.0f; movYC2 = 0.0f; movYC3 = 0.0f; movYC4 = 0.0f; movYC5 = 0.0f; movYC6 = 0.0f; movYC7 = 0.0f; movYC8 = 0.0f; movYC9 = 0.0f; movYS3 = 0.0f;
movYD1 = 0.0f; movYD2 = 0.0f; movYD3 = 0.0f; movYD4 = 0.0f; movYD5 = 0.0f; movYD6 = 0.0f; movYD7 = 0.0f; movYD8 = 0.0f; movYD9 = 0.0f;
```

Fig.22. Initialization of the variables to be used at zero.

After declaring all the variables earlier in the code, we began implementing the interaction of having the models rise while rotating. We did this with the help of the if statement. In the first if, as seen in **Fig. 23**, we compare if the square that needs to be activated is A1. If it is not the case, we move to the next square, after verifying that the dice-throwing key has not been pressed before the 3 seconds it takes for the model to rise. If this happens, we save the model again and assign new values to the variables that indicate a new dice roll will occur.

Next, in the following if, we simply assign the variable ultimaCasillaGato to A1. Then,

we check if the conditions are met for the model to rise, which are that the PuedeSubir variable has the value true, along with the avanzaGato flag. We also implemented this for the three-second wait during which the model must rise, and finally, we only raise the corresponding model with an increase in movement on the Y-coordinate.

```
// --- Casillas de la CHICAS SUPERPODEROSAS ---

//GATO
if (strcmp(casilla_Activacion, "A1") == 0) {
    strcpy_s(ultimaCasillaGato, sizeof(ultimaCasillaGato), "A1");
    if (PuedeSubir == true && avanzaGato) { // Subiendo
        if (movYA1 < 6.25f) { movYA1 += 0.3 * movOffsetHeli * deltaTime; rotA1 += 17.0 * deltaTime; }
        else { avanzaGato = false; ultimaActivacionGato = now; }
    }
    else if (!avanzaGato && (now - ultimaActivacionGato) >= 3.0f) { // Espera de 3 segundos
        if (movYA1 > 0.0f) { movYA1 -= 0.3 * movOffsetHeli * deltaTime; rotA1 -= 17.0 * deltaTime; }
    }
}
else if (strcmp(ultimaCasillaGato, "A1") == 0) { // Tirada de dados antes de los 3 segundos
    if (movYA1 > 0.0f) { movYA1 -= 0.3 * movOffsetHeli * deltaTime; rotA1 -= 17.0 * deltaTime; }
    else { strcpy_s(ultimaCasillaGato, sizeof(ultimaCasillaGato), ""); avanzaGato = true; }
}
}
```

Fig.23. Interaction of raising the model of the corresponding square in the Powerpuff Girls world.

```
// --- Casillas de BIG HERO 6 ---

//Hiro Hamada
if (strcmp(casilla_Activacion, "B4") == 0) {
    strcpy_s(ultimaCasillaHiroHamada_M, sizeof(ultimaCasillaHiroHamada_M), "B4");
    if (PuedeSubir == true && avanzaHiroHamada_M) {
        if (movYB4 < 12.25f) { movYB4 += 0.3 * movOffsetHeli * deltaTime; rotB4 += 13.2 * deltaTime; }
        else { avanzaHiroHamada_M = false; ultimaActivacionHiroHamada_M = now; }
    }
    else if (!avanzaHiroHamada_M && (now - ultimaActivacionHiroHamada_M) >= 3.0f) { // Espera de 3 segundos
        if (movYB4 > 0.0f) { movYB4 -= 0.3 * movOffsetHeli * deltaTime; rotB4 -= 13.2 * deltaTime; }
    }
}
else if (strcmp(ultimaCasillaHiroHamada_M, "B4") == 0) {
    if (movYB4 > 0.0f) { movYB4 -= 0.3 * movOffsetHeli * deltaTime; rotB4 -= 13.2 * deltaTime; }
    else { strcpy_s(ultimaCasillaHiroHamada_M, sizeof(ultimaCasillaHiroHamada_M), ""); avanzaHiroHamada_M = true; }
}
}
```

Fig.24. Interaction of raising the model of the corresponding square in the Big Hero 6 world.

```
// --- Casillas de PHINEAS Y FERB ---

//PHINEAS
if (strcmp(casilla_Activacion, "C7") == 0) {
    strcpy_s(ultimaCasillaPhineas, sizeof(ultimaCasillaPhineas), "C7");
    if (PuedeSubir == true && avanzaPhineas) {
        if (movYC7 < 6.55f) { movYC7 += 0.3 * movOffsetHeli * deltaTime; rotC7 += 16.6 * deltaTime; }
        else { avanzaPhineas = false; ultimaActivacionPhineas = now; }
    }
    else if (!avanzaPhineas && (now - ultimaActivacionPhineas) >= 3.0f) { // Espera de 3 segundos
        if (movYC7 > 0.0f) { movYC7 -= 0.3 * movOffsetHeli * deltaTime; rotC7 -= 16.6 * deltaTime; }
    }
}
else if (strcmp(ultimaCasillaPhineas, "C7") == 0) {
    if (movYC7 > 0.0f) { movYC7 -= 0.3 * movOffsetHeli * deltaTime; rotC7 -= 16.6 * deltaTime; }
    else { strcpy_s(ultimaCasillaPhineas, sizeof(ultimaCasillaPhineas), ""); avanzaPhineas = true; }
}
}
```

Fig.25. Interaction of raising the model of the corresponding square in the Phineas and Ferb world.

In this part of the code, we simply made the necessary adjustments for each of the models to be in the required position and scale next to the corresponding square with its respective texture, using transformation functions like `translate` to position the model correctly, `rotate` to face the correct direction, and `scale` to ensure all elements have a conventional and consistent size among them.

```
// --- MODELOS CHICAS SUPER PODEROSAS ---

model = modelaux; // Gato
model = glm::translate(model, glm::vec3(8.0f, movYA1 - 7.0f, 7.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotA1 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Gato.RenderModel();

model = modelaux; // Octi
model = glm::translate(model, glm::vec3(8.0f, movYA2 - 7.0f, 12.5f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotA2 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Octi.RenderModel();
```

Fig.26. Adjustment of the models corresponding to the Powerpuff Girls world.

```
// --- MODELOS BIG HERO 6 ---

model = glm::mat4(1.0); // Hiro Hamada
model = glm::translate(model, glm::vec3(-23.0f, movYB4 - 13.2f, 65.0f));
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotB4 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
HiroHamada_M.RenderModel();

model = glm::mat4(1.0); // Planta1
model = glm::translate(model, glm::vec3(-28.0f, movYB5 - 13.2f, 65.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotB5 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Planta1_M.RenderModel();
```

Fig.27. Adjustment of the models corresponding to the Big Hero 6 world.

```
// --- MODELOS PHINEAS Y FERB -----

model = modelaux; // Phineas
model = glm::translate(model, glm::vec3(-65.0f, movYC7 - 7.5f, 17.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotC7 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Phineas.RenderModel();

model = modelaux; // Ornitocyborg
model = glm::translate(model, glm::vec3(-65.0f, movYC8 - 7.5f, 12.5f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, rotC8 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.125f, 0.125f, 0.125f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ornitocyborg.RenderModel();
```

Fig.28. Adjustment of the models corresponding to the Phineas and Ferb world.

Journey on the board:

- The simultaneous roll of two dice will be implemented, one with 4 faces and one with 8 faces. The dice must start floating in the center of the board, fall, and upon making contact with the board, remain with a number always on the top face (simulating the real physics of rolling these two dice).

To implement the movement on the board based on rolling two dice, one with 8 faces and one with 4 faces, we first declare the variables that will help us perform the respective rotations of the dice to obtain each of the corresponding faces.

```
// Variables para rotacion de los Dados.
float movDado, movOffsetDado;
float RotacionX4, RotacionZ4, RotacionX8, RotacionZ8;
bool DadoRotando, DadoPiso;
```

Fig.29. Declaration of variables for the rotation of the dice.

Later, we create the dice with the help of triangles by creating a function specifically for this. So, the first thing we do is generate the indices. In this case, as shown in **Fig. 30**, we create the four-sided die with the displayed indices to form a pyramid, which is textured since, as we can see, we already have our texture coordinates obtained from the image we will use. Finally, we simply send it to CreateMesh, which will join the vertices to create our pyramid, and it will be saved in the meshList.

```

void Dado_4() {
    unsigned int dado4_indices[] = { 0,1,2, 3,4,5, 6,7,8, 9,10,11 };
    GLfloat dado4_vertices[] = {
        //Cara 1
        1.00f, 0.00f, 0.58f, 0.00f,0.00f, 0.00f, 1.00f, 0.00f, //0
        -1.00f, 0.00f, 0.58f, 1.00f,0.00f, 0.00f, 1.00f, 0.00f, //1
        0.00f, 0.00f, -1.15f, 0.50f,0.50f, 0.00f, 1.00f, 0.00f, //2
        //Cara 2
        1.00f, 0.00f, 0.58f, 1.00f,1.00f, 0.00f, -1.00f, -1.00f, //3
        -1.00f, 0.00f, 0.58f, 1.00f,0.00f, 0.00f, -1.00f, -1.00f, //4
        0.00f, 1.63f, 0.00f, 0.50f,0.50f, 0.00f, -1.00f, -1.00f, //5
        //Cara 3
        -1.00f, 0.00f, 0.58f, 0.00f,1.00f, 1.00f, -1.00f, 1.00f, //6
        0.00f, 0.00f, -1.15f, 1.00f,1.00f, 1.00f, -1.00f, 1.00f, //7
        0.00f, 1.63f, 0.00f, 0.50f,0.50f, 1.00f, -1.00f, 1.00f, //8
        //Cara 4
        1.00f, 0.00f, 0.58f, 0.00f,1.00f, -1.00f, -1.00f, 1.00f, //9
        0.00f, 0.00f, -1.15f, 0.00f,0.00f, -1.00f, -1.00f, 1.00f, //10
        0.00f, 1.63f, 0.00f, 0.50f,0.50f, -1.00f, -1.00f, 1.00f, //11
    };
    Mesh* dado4 = new Mesh();
    dado4->CreateMesh(dado4_vertices, dado4_indices, 96, 12);
    meshList.push_back(dado4);
}

```

Fig.30. Creation of the four-sided die.

In the same way that we created the four-sided die, we create the eight-sided die using polygons, which in this case are triangles. The only difference is clearly the number of declared indices, as having more faces creates a diamond-shaped figure, so the number of indices and vertices is greater. Similarly, we can see that we have the texture coordinates.

```

void Dado_8() {
    unsigned int dado8_indices[] = { 0,1,2, 3,4,5, 6,7,8, 9,10,11, 12,13,14, 15,16,17, 18,19,20, 21,22,23 };
    GLfloat dado8_vertices[] = {
        //Cara 1
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, 0.00f,-1.00f, -1.00f,//0
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 0.00f,-1.00f, -1.00f,//1
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, 0.00f,-1.00f, -1.00f,//2
        //Cara 2
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 1.00f,-1.00f, 0.00f,//3
        -1.00f, 0.00f,-1.00f, 0.00f,1.00f, 1.00f,-1.00f, 0.00f,//4
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, 1.00f,-1.00f, 0.00f,//5
        //Cara 3
        -1.00f, 0.00f,-1.00f, 0.00f,1.00f, 0.00f,-1.00f, 1.00f,//6
        1.00f, 0.00f,-1.00f, 1.00f,1.00f, 0.00f,-1.00f, 1.00f,//7
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, 0.00f,-1.00f, 1.00f,//8
        //Cara 4
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, -1.00f,-1.00f, 0.00f,//9
        1.00f, 0.00f,-1.00f, 1.00f,1.00f, -1.00f,-1.00f, 0.00f,//10
        0.00f, 1.51f, 0.00f, 0.50f,0.75f, -1.00f,-1.00f, 0.00f,//11
        //Cara 5
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, 0.00f, 1.00f,-1.00f,//12
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 0.00f, 1.00f,-1.00f,//13
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, 0.00f, 1.00f,-1.00f,//14
        //Cara 6
        -1.00f, 0.00f, 1.00f, 0.00f,0.50f, 1.00f, 1.00f, 0.00f,//15
        -1.00f, 0.00f,-1.00f, 0.00f,0.00f, 1.00f, 1.00f, 0.00f,//16
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, 1.00f, 1.00f, 0.00f,//17
        //Cara 7
        -1.00f, 0.00f,-1.00f, 1.00f,0.00f, 0.00f, 1.00f, 1.00f,//18
        1.00f, 0.00f,-1.00f, 0.00f,0.00f, 0.00f, 1.00f, 1.00f,//19
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, 0.00f, 1.00f, 1.00f,//20
        //Cara 8
        1.00f, 0.00f, 1.00f, 1.00f,0.50f, -1.00f, 1.00f, 0.00f,//21
        1.00f, 0.00f,-1.00f, 1.00f,0.00f, -1.00f, 1.00f, 0.00f,//22
        0.00f,-1.51f, 0.00f, 0.50f,0.25f, -1.00f, 1.00f, 0.00f,//23
    };
    Mesh* dado8 = new Mesh();
    dado8->CreateMesh(dado8_vertices, dado8_indices, 192, 24);
    meshList.push_back(dado8);
}

```

Fig.31. Creation of the eight-sided die.

Within our main, we call our functions for the creation of the dice, in addition to declaring and importing the textures that will be used on both dice, specifying the path where they are located.

```

CreateObjects();    Casillas_Esquina();    Casillas_Inter();
Dado_4();           Dado_8();           Centro();
CreateShaders();

camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.5f);

//Texturas para el piso
pisoTexture = Texture("Textures/piso.tga");           pisoTexture.LoadTextureA();
pisoTexture2 = Texture("Textures/CentroTableroSolo.tga");   pisoTexture2.LoadTextureA();
CentroTexture = Texture("Textures/CentroTablero.tga");       CentroTexture.LoadTextureA();

//Texturas de los Dados
Dado4_1 = Texture("Textures/Dado4_1.tga");           Dado4_1.LoadTextureA();
Dado4_2 = Texture("Textures/Dado4_2.tga");           Dado4_2.LoadTextureA();
Dado8 = Texture("Textures/Dado8.tga");               Dado8.LoadTextureA();

```

Fig.32. Importing dice textures.

We initialize the variables that we will use to achieve the rotation of the dice.

```
// Variables para rotacion de los Dados.
movDado = 0.0f;      movOffsetDado = 0.8f;
RotacionX4, RotacionZ4 = 0.0f; RotacionX8, RotacionZ8 = 0.0f;
DadoRotando = true;   DadoPiso = false;
```

Fig.33. Initialization of dice rotation variables.

To rotate the respective faces of the dice based on a random value when rolling the dice, we enter an if statement where, using the rand() function, we assign both dice a random number that will serve as the index in an array with the different faces belonging to each die. This allows us to select one of those faces from the array.

```
T_Transcurrido += deltaTime;
if (T_Transcurrido >= TIEMPO_CAMBIO) { FASE_01 = !FASE_01; T_Transcurrido = 0.0f; }

float angle = (T_Transcurrido / TIEMPO_CAMBIO) * 3.14159f;

// Calcular la dirección basada en el ángulo
float xDir = cos(angle);

//-- Movimiento para el tiro de los Dados --
if (mainWindow.gettiro() == true && DadoRotando == true) {
    movDado -= movOffsetDado * deltaTime;

    int sec_D4 = rand() % 4; //Giro para el Dado 4.
    Giro4 rot4 = Vista_Caras4[sec_D4];
    RotacionX4 = rot4.x; RotacionZ4 = rot4.z;
    mainWindow.setUltimoValorDado4(rot4.valor);

    int sec_D8 = rand() % 8; //Giro para el Dado 8.
    Giro8 rot8 = Vista_Caras8[sec_D8];
    RotacionX8 = rot8.x; RotacionZ8 = rot8.z;
    mainWindow.setUltimoValorDado8(rot8.valor);
}
```

Fig.34. Assignment of random dice rotation.

After rotating the dice when rolled, we sum the values we obtained so that this is the number of squares we need to move forward. We verify that we have reached the ground level in the roll, and then we store the amount obtained from each die face in the avance variable, as it represents the squares we will move.

Using a do-while loop, we perform the movement while decreasing the avance until it reaches zero, indicating we have reached the corresponding square. We then increase the position as the model will move based on its corresponding array of squares assigned when rolling the dice.

```

if (movDado < -24.0f) {
    DadoRotando = false;
    DadoPiso = true;
    PuedeSubir = false;
    avance = rot4.valor + rot8.valor;
    printf("\n - - - - - \n Los valores de los dados en el ultimo tiro fueron : \n");
    printf("Valor del Dado4: %.0f\n", rot4.valor);
    printf("Valor del Dado8: %.0f\n", rot8.valor);
    printf("Numero de Casillas a mover: %.0f\n", avance);

    do {
        if (Posicion == 40) { Posicion = 0; }
        PCasillas movcasilla = Mov_Casillas[Posicion];
        new_pos += movcasilla.mov;
        strcpy_s(casilla_Activacion, sizeof(casilla_Activacion), movcasilla.activacion); // Versión segura // Usar strcpy para copiar el string
        Posicion++; avance--;
        printf("Sumando, valor nuevo = %f\nEstamos en el arreglo = %d, de valor = %f\n", new_pos, Posicion, movcasilla.mov);
        printf("Nombre de la casilla: %s\n", casilla_Activacion);
    } while (avance != 0);
    printf("\n PROCEDEMOS CON EL MOVIMIENTO \n");
    Continuar = true;
}

if (mainWindow.getreinicio() == true && DadoPiso == true) { movDado = 0.0f; DadoRotando = true; DadoPiso = false; }

```

Fig.35. DO WHILE loop to calculate the total number of squares to move.

→ The avatar will walk on the board the number of squares equivalent to the number obtained with the dice.

For this activity, we declared various variables that help us control the movement and the position of the character: movx, movz, and giro, which are variables used to control the movement and rotation of the character.

```

// Variables para movimiento en el Tablero.
int Posicion;
float movX, movZ, giro, avance;
float movOffsetHeli, V_Rot_Helice;
float UbiCesq1, UbiCesq2, new_pos;
bool Cas1, Cas2, Cas3, Cas4, Continuar, PuedeSubir;
char casilla_Activacion[3];

bool PerPrincipal1, PerPrincipal2, PerPrincipal3;

```

Fig.36. Variables for character movement on the board.

```

// Variables para movimiento en el Tablero.
Posicion = 0;
movX, movZ, giro, avance = 0.0f;
movOffsetHeli = 0.5f; V_Rot_Helice = 0.0f;
UbiCesq1 = 54; UbiCesq2 = 54; new_pos = 0;
Cas1 = true; Cas2 = false; Cas3 = false; Cas4 = false; PuedeSubir = false;

PerPrincipal1 = true; PerPrincipal2 = false; PerPrincipal3 = false;

```

Fig.37. Initialization of variables to control the character's movement.

For more controlled movement of the character, we created the PCasillas structure, which helps us identify each square in an array of 40 pairs (value and name). These pairs assist us when advancing our character, as the first value is 0.

```

struct PCasillas { float mov; char activacion[3]; };
PCasillas Mov_Casillas[40] = {
    {7.0f, "A1"}, /*A1*/ {5.0f, "A2"}, /*A2*/ {5.0f, "A3"}, /*A3*/ {5.0f, "A4"}, /*A4*/ {5.0f, "A5"}, /*A5*/
    {5.0f, "A6"}, /*A6*/ {5.0f, "A7"}, /*A7*/ {5.0f, "A8"}, /*A8*/ {5.0f, "A9"}, /*A9*/ {7.0f, "S1"}, /*S2*/

    {7.0f, "B1"}, /*B1*/ {5.0f, "B2"}, /*B2*/ {5.0f, "B3"}, /*B3*/ {5.0f, "B4"}, /*B4*/ {5.0f, "B5"}, /*B5*/
    {5.0f, "B6"}, /*B6*/ {5.0f, "B7"}, /*B7*/ {5.0f, "B8"}, /*B8*/ {5.0f, "B9"}, /*B9*/ {7.0f, "S2"}, /*S3*/

    {7.0f, "C1"}, /*C1*/ {5.0f, "C2"}, /*C2*/ {5.0f, "C3"}, /*C3*/ {5.0f, "C4"}, /*C4*/ {5.0f, "C5"}, /*C5*/
    {5.0f, "C6"}, /*C6*/ {5.0f, "C7"}, /*C7*/ {5.0f, "C8"}, /*C8*/ {5.0f, "C9"}, /*C9*/ {7.0f, "S3"}, /*S4*/

    {7.0f, "D1"}, /*D1*/ {5.0f, "D2"}, /*D2*/ {5.0f, "D3"}, /*D3*/ {5.0f, "D4"}, /*D4*/ {5.0f, "D5"}, /*D5*/
    {5.0f, "D6"}, /*D6*/ {5.0f, "D7"}, /*D7*/ {5.0f, "D8"}, /*D8*/ {5.0f, "D9"}, /*D9*/ {7.0f, "S4"}, /*S1*/ };

```

Fig.38. Structure to control the movement units.

The functionality of the code is as follows: once the distance the character must travel is calculated, a series of ifs is executed, each checking if the character's position is before squares 1, 2, 3, and 4. If the character is in the position of the first corner, it checks if the distance to move is greater than the remaining distance. If that's the case, it activates the variable to trigger the model's "rise and fall" animation. Otherwise, indicating the character hasn't reached the position according to the dice value, the movZ variable is incremented. While incrementing, it checks that the distance does not exceed 54, which would indicate that the character has reached the corner square. If that's the case, it updates the current corner square to the next one, resets its position variables, and sets the character's giro (rotation) to -90°, preparing it for the next square. The other else if sequences follow the same procedure, but the movement is adjusted according to the direction the avatar should move. For the first corner, we move towards positive Z, for the second corner towards negative X, for the third corner towards negative Z, and upon reaching the fourth corner, which is the GO square, it also assigns values to flags that control the character change after each complete lap around the board.

```

if (Continuar = true) {
    if (Cas1 == true) {
        if (movZ >= new_pos) {PuedeSubir = true; }
        if (movZ < new_pos) {
            movZ += movOffsetHeli * deltaTime;
            if (movZ > 54) {
                Cas1 = false; Cas2 = true;
                new_pos = new_pos - 54;
                PuedeSubir = true;
                giro -= 90.0f;
                printf("\n C1Valor new_pos: %f\n", new_pos);
            }
        }
    }
    else if (Cas2 == true) {
        if (movX <= (-new_pos)) { PuedeSubir = true; }
        if (movX > (-new_pos)) {
            movX -= movOffsetHeli * deltaTime;
            if (movX < -54) {
                Cas2 = false; Cas3 = true;
                new_pos = new_pos - 54;
                PuedeSubir = true;
                giro -= 90.0f;
                printf("\n C2Valor new_pos: %f\n", new_pos);
            }
        }
    }
}

```

Fig.39. Sequence of ifs for movement depending on the side of the board where the character is located.

```

else if (Cas3 == true) {
    if (movZ <= (-new_pos + UbiCesq1)) { PuedeSubir = true; }
    if (movZ > (-new_pos + UbiCesq1)) {
        movZ -= movOffsetHeli * deltaTime;
        if (movZ < 1) {
            Cas3 = false; Cas4 = true;
            new_pos = new_pos - 54;
            PuedeSubir = true;
            giro -= 90.0f;
            printf("\n C3Valor new_pos: %f\n", new_pos);
        }
    }
    else if (movZ == (-new_pos + UbiCesq1)) { UbiCesq1 = movZ; }
}
else if (Cas4 == true) {
    if (movX >= (new_pos - UbiCesq2)) { PuedeSubir = true; }
    if (movX < (new_pos - UbiCesq2)) {
        movX += movOffsetHeli * deltaTime;
        if (movX > -1) {
            Cas4 = false; Cas1 = true; Cas2 = false; Cas3 = false;
            new_pos = new_pos - 54;
            PuedeSubir = true;
            giro -= 90.0f;
            printf("\n C4Valor new_pos: %f\n", new_pos);
            if (PerPrincipal1 == true) { PerPrincipal1 = false; PerPrincipal2 = true; PerPrincipal3 = false; }
            else if (PerPrincipal2 == true) { PerPrincipal1 = false; PerPrincipal2 = false; PerPrincipal3 = true; }
            else if (PerPrincipal3 == true) { PerPrincipal1 = true; PerPrincipal2 = false; PerPrincipal3 = false; }
        }
    }
    else if (movX == (new_pos - UbiCesq2)) { UbiCesq2 = movX; }
}
}

```

Fig.40. Sequence of ifs for movement depending on the side of the board where the character is located.

Once the movement direction for the axis the Avatar should move along is determined, these movement and rotation variables are assigned to the main character on the board.

```

model = glm::mat4(1.0f); // Personaje Principal en el tablero
model = glm::translate(model, glm::vec3(movX, 0.0f, movZ));
model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
if (PerPrincipal1 == true) {
    model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Perry.RenderModel(); }
else if (PerPrincipal2 == true) {
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon.RenderModel(); }

```

Fig.41. Assignment of movement for the main character according to the calculated distance.

The city:

- Around the board, there will be additional elements from the universes of the characters chosen by the students in the team, so that visually, it doesn't appear as though the board of the board game is floating in space.

The first step is declaring the variables of type Model, which will help us load the corresponding models into them. We also declare the file locations of the models.

```

// Modelos alrededor del tablero
Model Gato, Octi, Arbol, Burbuja, Bellota, Mojo, Peludito, Princesa, Profesor, Sedusa, Casa, Edificio, Alcaldia, Phineas, Ornitocborg,
Isabella, PeterPanda, Doofenshmirtz, TerryTortuga, Candace, Doofenshmirtz2D, Buford, PinkyChihuahua, Carl, Ferb,
HiroHamada_M, Planta1_M, AutoChico_M, GogoTomago_M, Motocicleta_M, GoldenGate_M, Planta2_M, BaymaxSuperArmadura_M, AutoGrande_M,
Fred_M, Planta3_M, CasaHiro_M, Megabot_M, Lampara, Lampara2, esf;

// Modelos ciudad
Model Edificio1, Edificio2, Edificio3, Edificio4, Edificio5, Edificio6;
Model Casa2, Doof, Edificio7, Edificio8, Edificio9, Edificio10, Edificio11, Edificio12, Edificio13, Edificio14;
Model Edificio15, Edificio16, Edificio17, Edificio18, Torre, Puente;

```

Fig.42. Variables for the models.

```

//Los 48 Modelos en el Tablero
Perry = Model();
Bombon = Model();
BaymaxBlanco_M = Model();
esf = Model();

Gato = Model();
Octi = Model();
Arbol = Model();
Burbuja = Model();
Bellota = Model();
Mojo = Model();
Peludito = Model();
Princesa = Model();
Profesor = Model();
Sedusa = Model();
Casa = Model();
Edificio = Model();
Alcaldia = Model();
HiroHamada_M = Model();
Planta1_M = Model();
AutoChico_M = Model();
GogoTomago_M = Model();
Motocicleta_M = Model();
GoldenGate_M = Model();
Planta2_M = Model();
BaymaxSuperArmadura_M = Model();

Perry.LoadModel("Models/Perry.obj");
Bombon.LoadModel("Models/bombon.obj");
BaymaxBlanco_M.LoadModel("Models/BaymaxBlanco/Bigmax_White_OBJ.obj");
esf.LoadModel("Models/esfera.obj");

Gato.LoadModel("Models/gato.obj");
Octi.LoadModel("Models/octi.obj");
Arbol.LoadModel("Models/arbol.obj");
Burbuja.LoadModel("Models/burbuja.obj");
Bellota.LoadModel("Models/bellota.obj");
Mojo.LoadModel("Models/mojo.obj");
Peludito.LoadModel("Models/peludito.obj");
Princesa.LoadModel("Models/princesa.obj");
Profesor.LoadModel("Models/profesor.obj");
Sedusa.LoadModel("Models/sedusa.obj");
Casa.LoadModel("Models/casa.obj");
Edificio.LoadModel("Models/edificio.obj");
Alcaldia.LoadModel("Models/alcaldia.obj");
HiroHamada_M.LoadModel("Models/HiroHamada/hiro.obj");
Planta1_M.LoadModel("Models/Planta1/Phormium_1.obj");
AutoChico_M.LoadModel("Models/AutoChico/isetta.obj");
GogoTomago_M.LoadModel("Models/GogoTomago/gogo.obj");
Motocicleta_M.LoadModel("Models/Motocicleta/motocicleta.obj");
GoldenGate_M.LoadModel("Models/GoldenGate/golden_gate_bridge.obj");
Planta2_M.LoadModel("Models/Planta2/hoewa_Forsteriana_1.obj");
BaymaxSuperArmadura_M.LoadModel("Models/BaymaxSuperArmadura/baymax_super.obj");

```

Fig.43. Loaded models of the characters.

```

AutoGrande_M = Model();
Fred_M = Model();
Planta3_M = Model();
CasaHiro_M = Model();
Megabot_M = Model();
Phineas = Model();
Ornitocyborg = Model();
Isabella = Model();
PeterPanda = Model();
Doofenshmirtz = Model();
TerryTortuga = Model();
Candace = Model();
Doofenshmirtz2D = Model();
Buford = Model();
PinkyChihuahua = Model();
Carl = Model();
Ferb = Model();
Lampara = Model();
Lampara2 = Model();

AutoGrande_M.LoadModel("Models/AutoGrande/500_fiat_lod_a.obj");
Fred_M.LoadModel("Models/Fred/fred.obj");
Planta3_M.LoadModel("Models/Planta3/Planta3.obj");
CasaHiro_M.LoadModel("Models/CasaHiro/CasaHiro.obj");
Megabot_M.LoadModel("Models/Megabot/MegaBot.obj");
Phineas.LoadModel("Models/PHINEAS.obj");
Ornitocyborg.LoadModel("Models/OrnitoCyborg.obj");
Isabella.LoadModel("Models/Isabella.obj");
PeterPanda.LoadModel("Models/PeterPanda.obj");
Doofenshmirtz.LoadModel("Models/Doofenshmirtz.obj");
TerryTortuga.LoadModel("Models/TerryTortuga.obj");
Candace.LoadModel("Models/Candace.obj");
Doofenshmirtz2D.LoadModel("Models/Doofenshmirtz2D.obj");
Buford.LoadModel("Models/Buford.obj");
PinkyChihuahua.LoadModel("Models/PinkyChihuahua.obj");
Carl.LoadModel("Models/Carl.obj");
Ferb.LoadModel("Models/Ferb.obj");
Lampara.LoadModel("Models/fobj_lamp_MODFIC.obj");
Lampara2.LoadModel("Models/obj_0401_street_lamp.obj");

```

Fig.44. Loaded models of the characters.

Similarly, for each of the buildings around the board that represent the city, the model is loaded into the variable to later be used with the corresponding geometric transformations.

```

// Edificios Ciudad
Edificio1 = Model();
Edificio1.LoadModel("Models/edificio1.obj");
Edificio2 = Model();
Edificio2.LoadModel("Models/edificio2.obj");
Edificio3 = Model();
Edificio3.LoadModel("Models/edificio3.obj");
Edificio4 = Model();
Edificio4.LoadModel("Models/edificio4.obj");
Edificio5 = Model();
Edificio5.LoadModel("Models/edificio5.obj");
Edificio6 = Model();
Edificio6.LoadModel("Models/edificio6.obj");

//Phineas and Ferb
Casa2 = Model();
Casa2.LoadModel("Models/casa2.obj");
Doof = Model();
Doof.LoadModel("Models/doof.obj");
Edificio7 = Model();
Edificio7.LoadModel("Models/edificio7.obj");
Edificio8 = Model();
Edificio8.LoadModel("Models/edificio8.obj");

Edificio9 = Model();
Edificio9.LoadModel("Models/edificio9.obj");
Edificio10 = Model();
Edificio10.LoadModel("Models/edificio10.obj");

//BIG HERO SIX
Edificio15 = Model();
Edificio15.LoadModel("Models/edificio15.obj");
Edificio16 = Model();
Edificio16.LoadModel("Models/edificio16.obj");
Edificio17 = Model();
Edificio17.LoadModel("Models/edificio17.obj");
Edificio18 = Model();
Edificio18.LoadModel("Models/edificio18.obj");
Torre = Model();
Torre.LoadModel("Models/torre.obj");
Puente = Model();
Puente.LoadModel("Models/puente.obj");

```

Fig.44 y Fig.45 . Loaded models of the buildings.

When we talk about transformations, we refer to the following, where each model of the city is assigned a translation, a rotation, and a scale if necessary to place each element in an appropriate location that does not interfere with any other model and ensures that all have a similar size.

```

//Edificio1
model = modelaux;
model = glm::translate(model, glm::vec3(92.0f, -0.8f, 30.0f));
//model = glm::translate(model, glm::vec3(42.0f, -0.8f, -20.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio1.RenderModel();

//Edificio2
model = modelaux;
model = glm::translate(model, glm::vec3(85.0f, -0.8f, -20.0f));
//model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio2.RenderModel();

//Alcaldia
model = modelaux;
model = glm::translate(model, glm::vec3(52.0f, -0.8f, 30.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Alcaldia.RenderModel();

```

Fig.46 . Location of the building models of the city of Saltadilla.

```

//PHINEAS Y FERB
//Casa
model = modelaux;
model = glm::translate(model, glm::vec3(-23.0f, -0.8f, -45.0f));
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Casa2.RenderModel();

//Doof casa
model = modelaux;
model = glm::translate(model, glm::vec3(-90.0f, -0.8f, -60.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Doof.RenderModel();

//Edificio7
model = modelaux;
model = glm::translate(model, glm::vec3(-85.0f, -0.8f, 5.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio7.RenderModel();

//Edificio8
model = modelaux;
model = glm::translate(model, glm::vec3(-87.0f, -0.8f, -25.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio8.RenderModel();

```

Fig.47 . Location of the building models of the Border Area..

```

// --- BIG HERO SIX -----
//Edificio15
model = modelaux;
model = glm::translate(model, glm::vec3(-35.0f, -0.8f, 80.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio15.RenderModel();

//Edificio16
model = modelaux;
model = glm::translate(model, glm::vec3(-57.0f, -0.8f, 80.0f));
model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio16.RenderModel();

//Torre
model = modelaux;
model = glm::translate(model, glm::vec3(-87.0f, -0.8f, 30.0f));
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Torre.RenderModel();

//Edificio18
model = modelaux;
model = glm::translate(model, glm::vec3(-87.0f, -0.8f, 55.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Edificio18.RenderModel();

```

Fig.48. Location of the building models of San Fran Tokyo.

Lighting:

- A point light will be used to illuminate the avatar during the night cycle and will be turned off during the day cycle.

For this procedure, we declare a point light, then we declare a sphere model in which we will place the point light, adjusting its position slightly behind the avatar and assigning it rotation variables. It is not necessary to specify movement variables since it is hierarchically linked to the main character model, so it will follow its path. An additional note is that if the model is Baymax, the light's position is raised to ensure visibility behind the character. In the case of the other two characters, Bombón and Perry, the light's position is lower.

```

pointLights[1] = PointLight(1.0f, 1.0f, 1.0f, 3.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.2f, 0.1f);
pointLightCount++; // LUZ DE SEGUIMIENTO.

```

Fig.49. Declaration of the follow light.

```

if (!FASE_01) {
    model = modelaux2; //Luz de seguimiento
    if (PerPrincipal3 == true) {
        model = glm::translate(model, glm::vec3(-20.0f, 70.0f, -20.0f));
        model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
        model = glm::scale(model, glm::vec3(0.8f, 0.8f, 0.8f));
    }
    else {
        model = glm::translate(model, glm::vec3(-4.0f, 11.0f, -4.0f));
        model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
        model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    }
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    esf.RenderModel();
    glm::vec3 lghtPrinc = glm::vec3(modelaux2[3]);
    pointLights[1].SetPosition(lghtPrinc.x, lghtPrinc.y, lghtPrinc.z);
    activeLights[activeLightCount++] = pointLights[1];
}

```

Fig.50. Declaration of the tracking light.

- There will be 4 light sources at each corner of the board, consisting of a mix of point and positional lights, which will turn on during the night cycle if the avatar passes through that square.

For this implementation, we chose 3 spotlights for the lights and one point light for the last one. We also declared some streetlamp models to better implement these lights. The declaration of the models is the same, accompanied by the specific position of the point light or spotlight. To ensure that this turns on correctly, we declare an 'if' statement for turning on and an 'else' statement for turning off in each case. In the case of the point light, we simply indicate that the information for turning it on is sent once when it is night through the flag FASE_01. Meanwhile, for the spotlights, we need to indicate the sending of information for turning them on, but nothing is sent for turning them off.

```
unsigned int pointLightCount = 0;
pointLights[0] = PointLight(1.0f, 1.0f, 1.0f, 5.0f, 3.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.2f, 0.1f);
pointLightCount++; // LUZ ESQUINA 01.

pointLights[1] = PointLight(1.0f, 1.0f, 1.0f, 3.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.2f, 0.1f);
pointLightCount++; // LUZ DE SEGUIMIENTO.

unsigned int spotLightCount = 0;

spotLights[0] = SpotLight(1.0f, 1.0f, 1.0f, 4.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.1f, 0.05f, 45.0f);
spotLightCount++; // LUZ ESQUINA 02.

spotLights[1] = SpotLight(1.0f, 1.0f, 1.0f, 4.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.1f, 0.05f, 45.0f);
spotLightCount++; // LUZ ESQUINA 03.

spotLights[2] = SpotLight(1.0f, 1.0f, 1.0f, 4.0f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.1f, 0.05f, 45.0f);
spotLightCount++; // LUZ ESQUINA 04.
```

Fig.51. Arrangements of point lights and spotlights.

```
//Lampara 01
model = modelLamp1;
model = glm::translate(model, glm::vec3(8.0f, 0.2f, -8.0f));
model = glm::scale(model, glm::vec3(0.15f, 0.15f, 0.15f));
modelLamp1 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara.RenderModel();
glm::vec3 lamparaPos1 = glm::vec3(modelLamp1[3]);
pointLights[0].SetPosition(lamparaPos1.x, lamparaPos1.y + 8.2f, lamparaPos1.z);
if (!FASE_01) { activeLights[activeLightCount++] = pointLights[0]; }

//Lampara 02
model = modelLamp2;
model = glm::translate(model, glm::vec3(6.0f, 0.2f, 62.0f));
model = glm::rotate(model, -135 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
modelLamp2 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara2.RenderModel();
glm::vec3 lamparaPos2 = glm::vec3(modelLamp2[3]);
glm::vec3 lightPos = glm::vec3(lamparaPos2.x - 3.2f, lamparaPos2.y + 11.6f, lamparaPos2.z - 3.2f);
if (!FASE_01) { spotLights[0].SetFlash(lightPos, glm::vec3(0.0f, -1.0f, 0.0f)); } // Noche - Luz encendida
else { spotLights[0].SetFlash(lightPos, glm::vec3(0.0f)); } // Dia - Luz apagada
```

Fig.52. Position of the lamps and on/off cycle.

```

//Lampara 04
model = modelLamp4;
model = glm::translate(model, glm::vec3(-62.0f, 0.2f, -6.0f));
model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.25f, 0.25f, 0.25f));
modelLamp4 = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Lampara2.RenderModel();
glm::vec3 lamparaPos4 = glm::vec3(modelLamp4[3]);
glm::vec3 lightPos3 = glm::vec3(lamparaPos4.x + 3.2f, lamparaPos4.y + 11.6f, lamparaPos4.z + 3.2f);
if (!FASE_01) { spotLights[2].SetFlash(lightPos3, glm::vec3(0.0f, -1.0f, 0.0f)); } // Noche - Luz encendida
else { spotLights[2].SetFlash(lightPos3, glm::vec3(0.0f)); } // Dia - Luz apagada

shaderList[0].SetPointLights(activeLights, activeLightCount);

```

Fig.53. Position of the lamps and the on/off cycle..

→ There will be a directional light that represents the sun, which will 'move' across the stage in an East-West direction.

To begin with this declaration, we first generate the 2 lights that represent day and night. Then, we create a sequence to measure the elapsed time, in this case giving a total of approximately 16 seconds for the transition between day and night. Additionally, to achieve a better effect in the movement of the sun, we calculate the direction angle on the x-axis that will represent the sun's movement, using a cosine function based on the elapsed time and the transition time, but also multiplied by pi to enhance the movement.

```

LightDAY = DirectionalLight (1.0f, 1.0f, 0.9f, 0.4f, 0.6f, 0.0f, -1.0f, 0.0f);
LightNIGHT = DirectionalLight (0.8f, 0.8f, 1.0f, 0.1f, 0.2f, 0.0f, -1.0f, 0.0f);

```

Fig.54. Declaration of Day and Night Lights.

```

T_Transcurrido += deltaTime;
if (T_Transcurrido >= TIEMPO_CAMBIO) { FASE_01 = !FASE_01; T_Transcurrido = 0.0f; }

float angle = (T_Transcurrido / TIEMPO_CAMBIO) * 3.14159f;

// Calcular la dirección basada en el ángulo
float xDir = cos(angle);

```

Fig.55. Calculation of the Day and Night Light Cycle.

Likewise, once the time has elapsed, the flag FASE_01 is set to true, indicating that it is daytime; otherwise, it indicates nighttime lighting. To control this change, an 'if' statement was declared that executes based on the value of this flag. In both cases, the calculated direction value on the X-axis is applied. The only real difference between both declarations is the point light and the skybox that changes with it

```

if (FASE_01) {
    LightDAY.SetDirection(xDir, -0.25f, 0.0f/*zDir*/);
    shaderList[0].SetDirectionalLight(&LightDAY);
    skyboxDia.DrawSkybox(camera.calculateViewMatrix(), projection);
}
else {
    LightNIGHT.SetDirection(xDir, -0.25f, 0.0f/*zDir*/);
    shaderList[0].SetDirectionalLight(&LightNIGHT);
    skyboxNoche.DrawSkybox(camera.calculateViewMatrix(), projection);
}

```

Fig.56. Day or Night Lighting Sequence.

Animation:

- Around the board, in what represents the city, there will be other avatars of the team members performing a characteristic animation of their character (it cannot be a walking animation).

Additional Points:

Avatar Change on the Board: It is implemented so that each time the avatar crosses the starting square (an indicated corner), it changes to one of the avatars of the other team members.

To implement this avatar change, we create an 'else if' sequence to change the position with each character assignment, all through the flags PerPrincipal, which are assigned a value during the last movement sequence of the character. The sequence will assign values to each flag depending on which one is active first, starting with PerPrincipal1, which belongs to Perry.

The element with the most declarations is Baymax, as it is the only character that performs an animation while outside the board.

```

else if (Cas4 == true) {
    if (movX >= (new_pos - UbiCesq2)) { PuedeSubir = true; }
    if (movX < (new_pos - UbiCesq2)) {
        movX += movOffsetHeli * deltaTime;
        if (movX > -1) {
            Cas4 = false; Cas1 = true; Cas2 = false; Cas3 = false;
            new_pos = new_pos - 54;
            PuedeSubir = true;
            giro -= 90.0f;
            printf("\n C4Valor new_pos: %f\n", new_pos);
            if (PerPrincipal1 == true) { PerPrincipal1 = false; PerPrincipal2 = true; PerPrincipal3 = false; }
            else if (PerPrincipal2 == true) { PerPrincipal1 = false; PerPrincipal2 = false; PerPrincipal3 = true; }
            else if (PerPrincipal3 == true) { PerPrincipal1 = true; PerPrincipal2 = false; PerPrincipal3 = false; }
        }
    }
    else if (movX == (new_pos - UbiCesq2)) { UbiCesq2 = movX; }
}

```

Fig.57. Activation of Flags for Avatar Position Change.

According to the aforementioned flags, it is possible that each time one of these is activated, the model of each of the main characters: Perry the Platypus, Bombón, and Baymax will swap positions at the starting position of the board until, after crossing the GO square, the next model in the list simulates the rolls

```

model = glm::mat4(1.0f); // Personaje Principal en el tablero
model = glm::translate(model, glm::vec3(movX, 0.0f, movZ));
model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
if (PerPrincipal1 == true) {
    model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Perry.RenderModel(); }
else if (PerPrincipal2 == true) {
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon.RenderModel(); }

```

Fig.58. Rendering of the models according to the flags on the board.

```

else if (PerPrincipal3 == true) {
    model = glm::mat4(1.0); // Baymax Blanco Con Jerarquía
    model = glm::translate(model, glm::vec3(movX, 0.0f, movZ));
    model = glm::rotate(model, glm::radians(giro), glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BaymaxBlanco_M.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-12.0f, 60.0f, 0.0f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BrazoDerechoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-13.0f, -30.0f, 0.0f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    ManoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-5.0f, -5.0f, 0.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoMedioBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-5.0f, -4.0f, 2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoIzquierdoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-5.0f, -4.0f, -2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoDerechoBaymax.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(2.0f, -4.0f, 2.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    DedoGordoBaymax.RenderModel();
}

```

Fig.59. Rendering of the Baymax model according to the flags on the board.

```

model = modelaux; // Personaje Secundario1 afuera del tablero
model = glm::translate(model, glm::vec3(8.0f, -0.8f, 0.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));

if (PerPrincipal1 == true) {
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon.RenderModel();
}

else if (PerPrincipal2 == true) {
    model = glm::mat4(1.0); // Baymax Animado
    model = glm::translate(model, glm::vec3(8.0f, -0.8f, 0.0f));
    model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BaymaxBlanco_M.RenderModel();

    model = modelauxBaymax;
    model = glm::translate(model, glm::vec3(-12.0f, 60.0f, 0.0f));
    model = glm::rotate(model, giroBrazo * toRadians, glm::vec3(-1.0f, 0.0f, 0.0f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BrazoDerechoBaymax.RenderModel();

    model = modelauxBaymax;
    model = glm::translate(model, glm::vec3(-13.0f, -30.0f, 0.0f));
    model = glm::rotate(model, giraMano * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    ManoBaymax.RenderModel();
}

```

Fig.60. Rendering of the model according to the flags outside the board.

```

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(-5.0f, -4.0f, 0.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoMedioBaymax.RenderModel();

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(-5.0f, -4.0f, 2.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoIzquierdoBaymax.RenderModel();

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(-5.0f, -4.0f, -2.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoDerechoBaymax.RenderModel();

model = modelauxBaymax;
model = glm::translate(model, glm::vec3(2.0f, -4.0f, 2.0f));
model = glm::rotate(model, sin(giraDedos) * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
DedoGordoBaymax.RenderModel();
}

```

Fig.61. Rendering of the model according to the flags outside the board.

```

model = glm::mat4(1.0); // Personaje Secundario2 afuera del tablero
model = glm::translate(model, glm::vec3(0.0f, -0.8f, -8.0f));
if (PerPrincipal1 == true) {
    model = glm::mat4(1.0); // Baymax Animado
    model = glm::translate(model, glm::vec3(0.0f, -0.8f, -8.0f));
    model = glm::scale(model, glm::vec3(0.1f, 0.1f, 0.1f));
    modelauxBaymax = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BaymaxBlanco_M.RenderModel();
}

```

Fig.62. Rendering of the model according to the flags outside the board.

```

}
else if (PerPrincipal2 == true) {
    model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Perry.RenderModel();
}
else if (PerPrincipal3 == true) {
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon.RenderModel();
}
}

```

Fig.63. Rendering of the model according to the flags outside the board.

Baymax Animation

One of the main characters has an animation while waiting to be selected to play; this animation is the classic greeting between Baymax and Hiro Hamada in the movie "Big Hero 6." To perform the animation, there is a boolean flag named `mueveBrazo`, which indicates whether Baymax's arm should rise or not. The arm lifts until it reaches a position of 85° , where it stops and activates another boolean flag called `mueveMano`, indicating that the hand should rotate to make a fist bump. Once it has rotated and is in the correct position, the `mueveBrazo` flag is reactivated to complete the greeting. After performing all these actions, the steps repeat so that the character is continuously greeting until it is their turn to play.

```

//Animacion BAYMAX "SALUDO"
if (mueveBrazo) {
    if (giroBrazo < 85 or giroBrazo > 90) {
        giroBrazo += giroBrazoOffset * deltaTime;
    }
    else { mueveBrazo = !mueveBrazo; mueveMano = true; }
    if (giroBrazo > 180) { giroBrazo = 0.0; giraDedos = 0.0; giraMano = 0.0; }
}
else {
    if (mueveMano) {
        if (giraMano < 90) { giraMano += giraManoOffset * deltaTime; }
        else {
            giroBrazo = 91; mueveMano = -!mueveMano;
            giraDedos += 0.5f * deltaTime; mueveBrazo = true;
        }
    }
}
}

```

Fig.64. Lines to Animate Baymax.

Blossom Animation

Another main character present on the board scene is Blossom, who has his own animation. First, the variables for movement along the different axes were defined to move the character, along with the rotation for the arm to create the animation of it rising, and the variables for the right leg to also go up on the y-axis, achieving the pose of flying. Additionally, there are respective offsets and the time used for the function that allows Blossom to fly in a heart-shaped trajectory, as well as `deltaT` to

adjust the speed at which he completes the trajectory.

```
// VARIABLES PARA ANIMACIÓN DE BOMBON
float movBx, movBy, movBz, rotB;
float movOffset, movOffsetp, movOffsetb;
float rotbombonOffset, rotbrazobomOffset;
float t, deltaT;
float subiry, rotbrazo, bajarbrazo;

Model Bombon2, BrazoI, PiernaD;
```

Fig.65. Variables and Models for Blossom's Animation.

```
// Modelo de Bombon por partes
Bombon2 = Model();          Bombon2.LoadModel("Models/bombon2.obj");
BrazoI = Model();           BrazoI.LoadModel("Models/brazoI.obj");
PiernaD = Model();          PiernaD.LoadModel("Models/piernaD.obj");
```

Fig.66. Declaration and Path of Where Blossom's Models Are Located.

```
// Variables de movimeinto para Bombon
movBx = 0.0f; movBy = 0.0f; movBz = 0.0f; rotB = 0.0f; rotbrazo = 0.0f;
movOffset = 0.03f; movOffsetp = 0.0003f; rotbombonOffset = 0.02f;
subiry = 0.0f; rotbrazobomOffset = 0.04f; movOffsetb = 0.0004f; bajarbrazo = 0.0f;
t = 0.0f; deltaT = 0.01f;
```

Fig.67. Initialization of Blossom's Variables.

For Blossom's animation, we first used `rotbrazo` to increase the counter until it reached an angle of -120 degrees. Subsequently, to better position the raised arm, it was also necessary to lower it a few centimeters, so it was positioned 0.5 units down. Finally, Blossom rises along the y-axis until reaching a height of 30, where he will begin to trace a heart-shaped trajectory using the function $16 * \text{pow}(\sin(t), 3)$ for movement in x and the function $-25.0f + (13 * \cos(t) - 5 * \cos(2 * t) - 2 * \cos(3 * t) - \cos(4 * t))$ for movement in z, increasing the variable `t`, when `t` equals 2π , the variable `t` resets.

```
// Animacion de Bombon
if (rotbrazo > -120.0f) { rotbrazo -= rotbrazobomOffset * deltaTime; }
if (bajarbrazo > -0.5f) { bajarbrazo -= movOffsetb * deltaTime; }
if (movBy < 30.0f) { movBy += movOffset * deltaTime;
    if (rotB < 30) { rotB += rotbombonOffset * deltaTime; }
    if (subiry < 0.7f) { subiry += movOffsetp * deltaTime; } }
else { movBx = (16 * pow(sin(t), 3));
    movBz = -25.0f + (13 * cos(t) - 5 * cos(2 * t) - 2 * cos(3 * t) - cos(4 * t));
    t += deltaT;
    if (t > 2 * M_PI) t = 0.0;}
```

Fig.68. Animation of Blossom.

```

if (PerPrincipal1 == true) {
    model = glm::mat4(1.0f);
    model = glm::rotate(model, rotB * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    model = glm::translate(model, glm::vec3(65.0f + movBx, -0.8f + movBy, movBz));
    model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
    modelaux2 = model;
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bombon2.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(0.2f, 4.0f + bajarbrazo, -0.8f));
    model = glm::rotate(model, rotbrazo * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    BrazoI.RenderModel();

    model = modelaux2;
    model = glm::translate(model, glm::vec3(-0.3f, 2.3f + subiry, -0.5f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    PiernaD.RenderModel();
}

```

Fig.69. Positioning of the models of Blossom.

Cameras:

→ A third-person camera linked to the avatar.

For the third-person camera, the first step was to declare the variables to be used, as shown in Fig. 70. These include a position vector, the distance, and the height at which the third-person camera is located. We also declared the methods that we will use to switch between cameras.

```

// Variables para cámara en tercera persona
glm::vec3 targetPosition;
float cameraDistance;
float cameraHeight;

```

Fig.70. Variables for the third-person camera.

```

// Métodos para cambiar entre modos de cámara
void toggleAerialView();
bool isInAerialView() { return isAerialView; }

glm::vec3 getCameraPosition();
glm::vec3 getCameraDirection();
glm::mat4 calculateViewMatrix();

```

Fig.71. Methods for switching between cameras.

In the Camera function, we initialize the previously declared variables and then update the camera by obtaining the character's position coordinates and the angle in radians, so that the camera has the same rotation as the character. Finally, we position the camera behind the character so that it follows them.

```

Camera::Camera(glm::vec3 startPosition, glm::vec3 startUp, GLfloat startYaw)
{
    position = startPosition;
    worldUp = startUp;
    yaw = startYaw;
    pitch = startPitch;
    front = glm::vec3(0.0f, 0.0f, -1.0f);

    moveSpeed = startMoveSpeed;
    turnSpeed = startTurnSpeed;

    // Inicializar valores para tercera persona
    cameraDistance = 8.0f;    // Ajusta según necesites
    cameraHeight = 8.0f;     // Ajusta según necesites

    // Inicializar valores para vista aérea
    isAerialView = false;
    aerialHeight = 500.0f;    // Altura de la vista aérea
    aerialMoveSpeed = 0.5f;   // Velocidad de movimiento en vista aérea

    update();
}

```

Fig.72. Function for the third-person camera.

```

// Implementación del método updateThirdPerson
void Camera::updateThirdPerson(const glm::vec3& characterPosition, float characterRotation) {
    // Actualizar la posición objetivo (personaje)
    targetPosition = characterPosition;

    // Calcular la posición de la cámara basada en la rotación del personaje
    float angleInRadians = glm::radians(characterRotation);

    // Posicionar la cámara detrás del personaje
    position.x = targetPosition.x - (cameraDistance * sin(angleInRadians));
    position.z = targetPosition.z - (cameraDistance * cos(angleInRadians));
    position.y = targetPosition.y + cameraHeight;

    // Hacer que la cámara mire hacia el personaje
    front = glm::normalize(targetPosition - position);

    // Actualizar los vectores right y up
    right = glm::normalize(glm::cross(front, worldUp));
    up = glm::normalize(glm::cross(right, front));
}

```

Fig.73. Function to update the third-person camera.

Finally, if the camera is to be switched from third-person to aerial view, the last position is saved before making the change. To carry out the switch, we declare the variables from Fig. 74 and initialize them as false. Then, we check if the V key has been pressed and whether it is indicated as being pressed or not. If this is true, the camera and projection are updated accordingly.

```

void Camera::toggleAerialView() {
    isAerialView = !isAerialView;
    if (isAerialView) { // Guardar la posición actual antes de cambiar a vista aérea
        position.y = aerialHeight; // Apuntar directamente hacia abajo
        front = glm::vec3(0.0f, -1.0f, 0.0f);
        right = glm::vec3(1.0f, 0.0f, 0.0f);
        up = glm::vec3(0.0f, 0.0f, -1.0f);
    }
}

```

```
bool useOrthoProjection = false;
bool vKeyPressed = false;
```

```
if (mainWindow.getKeys()[GLFW_KEY_V] && !vKeyPressed) { // Cambiar entre vistas con tecla V
    camera.toggleAerialView();
    useOrthoProjection = camera.isInAerialView();
    vKeyPressed = true;
}
if (!mainWindow.getKeys()[GLFW_KEY_V]) {vKeyPressed = false;}

// Actualizar Cámara
if (camera.isInAerialView()) {camera.updateAerialView(mainWindow.getKeys(), deltaTime);}
else {camera.updateThirdPerson(glm::vec3(movX, 0.0f, movZ), giro);}

// Actualizar proyección
glm::mat4 projection;
if (useOrthoProjection) {
    float aspectRatio = (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight();
    float orthoSize = 100.0f;
    projection = glm::ortho(-orthoSize * aspectRatio, orthoSize * aspectRatio, -orthoSize, orthoSize, 0.1f, 1000.0f);}
else {projection = glm::perspective(45.0f, (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 1000.0f);}
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // Actualizar el shader con la nueva matriz de proyección
```

Fig.74. Function and variables to switch to third-person camera.

→ An aerial camera that will allow viewing the entire city from above and can only move in an orthogonal manner.

```
// Variables para cámara aérea
bool isAerialView;
float aerialHeight;
float aerialMoveSpeed;
```

Fig.75. Variables for the aerial camera.

Similarly, in the Camera function, the variables for the aerial camera are initialized by setting the isAerialView variable to false, the height to 500, and the speed at which the view will move orthogonally to 0.5.

```
Camera::Camera(glm::vec3 startPosition, glm::vec3 startUp, GLfloat startYaw)
{
    position = startPosition;
    worldUp = startUp;
    yaw = startYaw;
    pitch = startPitch;
    front = glm::vec3(0.0f, 0.0f, -1.0f);

    moveSpeed = startMoveSpeed;
    turnSpeed = startTurnSpeed;

    // Inicializar valores para tercera persona
    cameraDistance = 8.0f; // Ajusta según necesites
    cameraHeight = 8.0f; // Ajusta según necesites

    // Inicializar valores para vista aérea
    isAerialView = false;
    aerialHeight = 500.0f; // Altura de la vista aérea
    aerialMoveSpeed = 0.5f; // Velocidad de movimiento en vista aérea

    update();
}
```

Fig.76. Function for the aerial camera.

In this function, we calculate the speed at which the camera will move and store the

keys used to move the camera orthogonally, such as W, S, A, and D. We then assign the value of the height to the camera's position vector along the Y-axis.

```
void Camera::updateAerialView(bool* keys, GLfloat deltaTime) {
    if (!isAerialView) return;
    GLfloat velocity = aerialMoveSpeed * deltaTime;

    // Movimiento ortogonal en el plano XZ
    if (keys[GLFW_KEY_W]) { position.z -= velocity; } // Mover hacia adelante en Z
    if (keys[GLFW_KEY_S]) { position.z += velocity; } // Mover hacia atrás en Z
    if (keys[GLFW_KEY_A]) { position.x -= velocity; } // Mover hacia la izquierda en X
    if (keys[GLFW_KEY_D]) { position.x += velocity; } // Mover hacia la derecha en X
    position.y = aerialHeight; // Mantener la altura constante
}
```

Fig.77. Function to update the aerial camera.

Finally, each time the letter V is pressed, the camera changes either to the aerial or dynamic view, depending on which one is active at the moment the change is triggered. If the letter is pressed and the camera was previously in third-person view, it switches to the aerial view as shown in Fig. 78, updating the camera and projection accordingly.

```
if (mainWindow.getKeys()[GLFW_KEY_V] && !vKeyPressed) { // Cambiar entre vistas con tecla V
    camera.toggleAerialView();
    useOrthoProjection = camera.isInAerialView();
    vKeyPressed = true;
}
if (!mainWindow.getKeys()[GLFW_KEY_V]) { vKeyPressed = false; }

// Actualizar Cámara
if (camera.isInAerialView()) { camera.updateAerialView(mainWindow.getKeys(), deltaTime); }
else { camera.updateThirdPerson(glm::vec3(movX, 0.0f, movZ), giro); }

// Actualizar proyección
glm::mat4 projection;
if (useOrthoProjection) {
    float aspectRatio = (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight();
    float orthoSize = 100.0f;
    projection = glm::ortho(-orthoSize * aspectRatio, orthoSize * aspectRatio, -orthoSize, orthoSize, 0.1f, 1000.0f);
}
else { projection = glm::perspective(45.0f, (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 1000.0f); }

glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection)); // Actualizar el shader con la nueva matriz de proyección
```

Fig.78. Change and update of the aerial camera.

Audio:

→ Continuous soundtrack audio that will always play in a loop.

To include soundtrack-type audio in our project, we used the irrKlang audio library. To achieve this, we configured our project by adding the files from the include and lib folders. Subsequently, we declared the lines shown in Fig. 79 at the top of the code to enable the use of the library.

```
#include<iostream>
#include <irrKlang.h>
using namespace irrklang;
#pragma comment(lib, "irrKlang.lib")
```

Fig.79. Declaración para la librería de audio.

We add the variable to turn the music on or off, defining it as a boolean set to true. Then, the function `musicaFondo` is created, and if the music variable is enabled, the sound engine is initialized with default parameters using the `createIrrKlangDevice()` function. It verifies if the engine started correctly and calls `play2D`, passing the path to the audio file we want to play as the first parameter and enabling loop playback as the second parameter. Finally, the `musicaFondo` function is simply called within the main.

```
//Musica  
bool On = true;
```

```
int musicaFondo() {  
    if (On) {  
        On = false;  
        ISoundEngine* engine = createIrrKlangDevice();  
        if (!engine) {return 0;} engine->play2D("audio/doodle.mp3", true);  
        return 0;  
    }  
}
```

Fig.80. Function for background music.

```
int main() {  
    mainWindow = Window(1366, 768); // 1280, 1024 or 1024, 768  
    mainWindow.Initialise();  
  
    CreateObjects();    Casillas_Esquina();    Casillas_Inter();  
    Dado_4();           Dado_8();             Centro();  
    CreateShaders();    musicaFondo();  
}
```

Fig.81. Call to the function in the main.

Conclusiones:

Barragán Pilar Diana

The development of this project allowed me to understand the complexity and dedication involved in designing interactive 3D environments. The initial planning, based on our guide document, was crucial for defining the main elements and functionalities. I was surprised by how much the management of textures and lights influenced the final visual experience, especially the use of pointlights and spotlights to add realism to the scenes. This work also highlighted the importance of collaboration and how the continuous adjustment of our ideas was essential to overcoming technical obstacles.

Medrano Miranda Daniel Ulises

Working on this project was an eye-opening experience that strengthened my skills in programming and 3D graphics management. Implementing functions like dice rolling and avatar movement on the board showed me how crucial precision is in physical simulation. Additionally, the use of the irrKlang library for music added a very satisfying layer of immersion to the environment. The process of perfecting every detail, from sketches in Geogebra to the integration of animated models, made me appreciate the value of refinement and continuous optimization in technical development.

Uriarte Ortiz Enrique Yahir

The completion of this project was a great challenge that combined creativity and technical knowledge. The detailed organization of 3D models and the implementation of lights and textures, based on our original documentation, required meticulous planning. I was struck by how each element, from the daytime and nighttime texture changes to the animation of characters like Baymax, contributed to a more dynamic and engaging environment. Additionally, I learned that teamwork and the ability to adapt to changes in requirements are essential for the success of any ambitious project.